
Rerank와 Modular RAG

2. Rerank

목차

1. Information Retrieval

2. Reranking

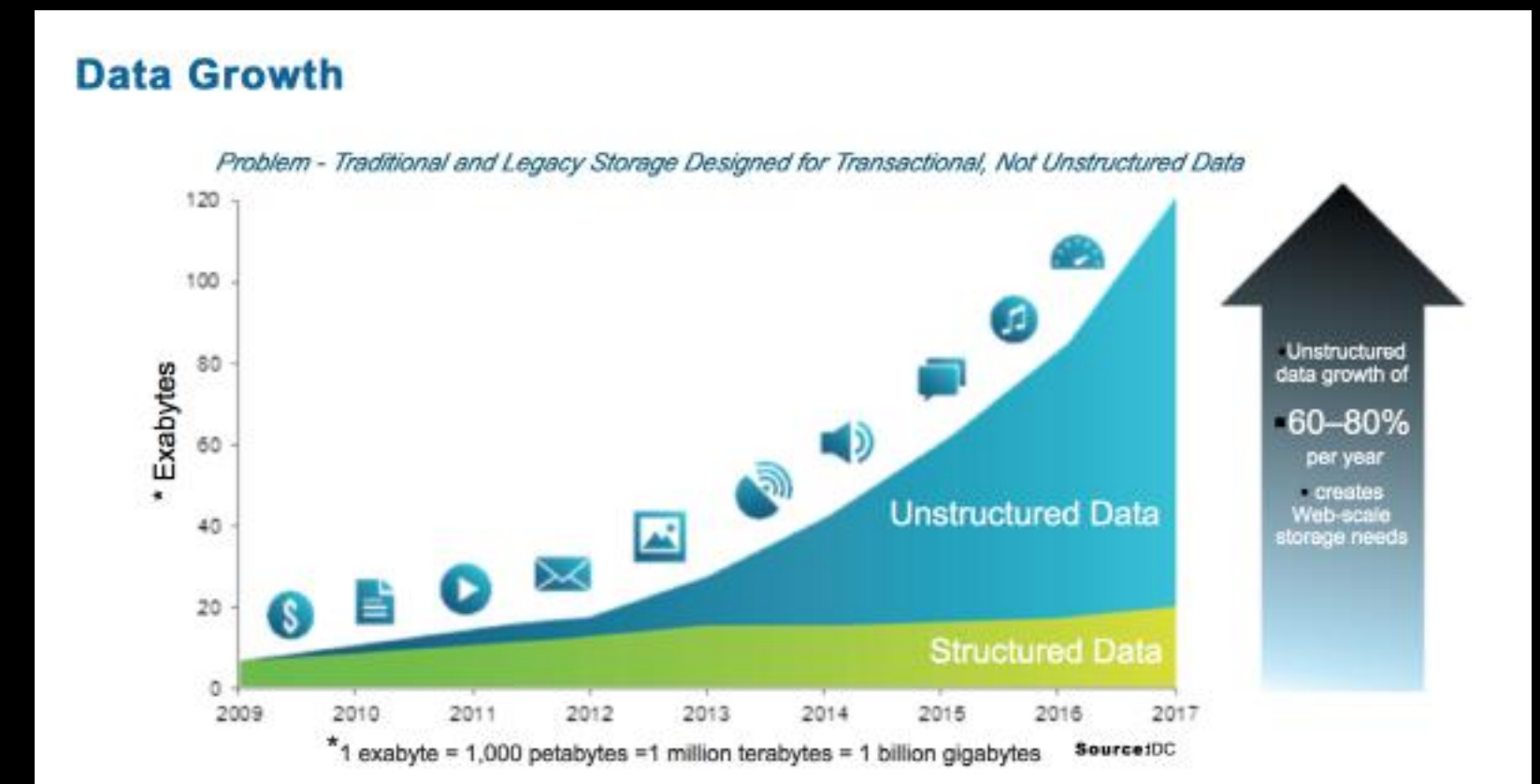
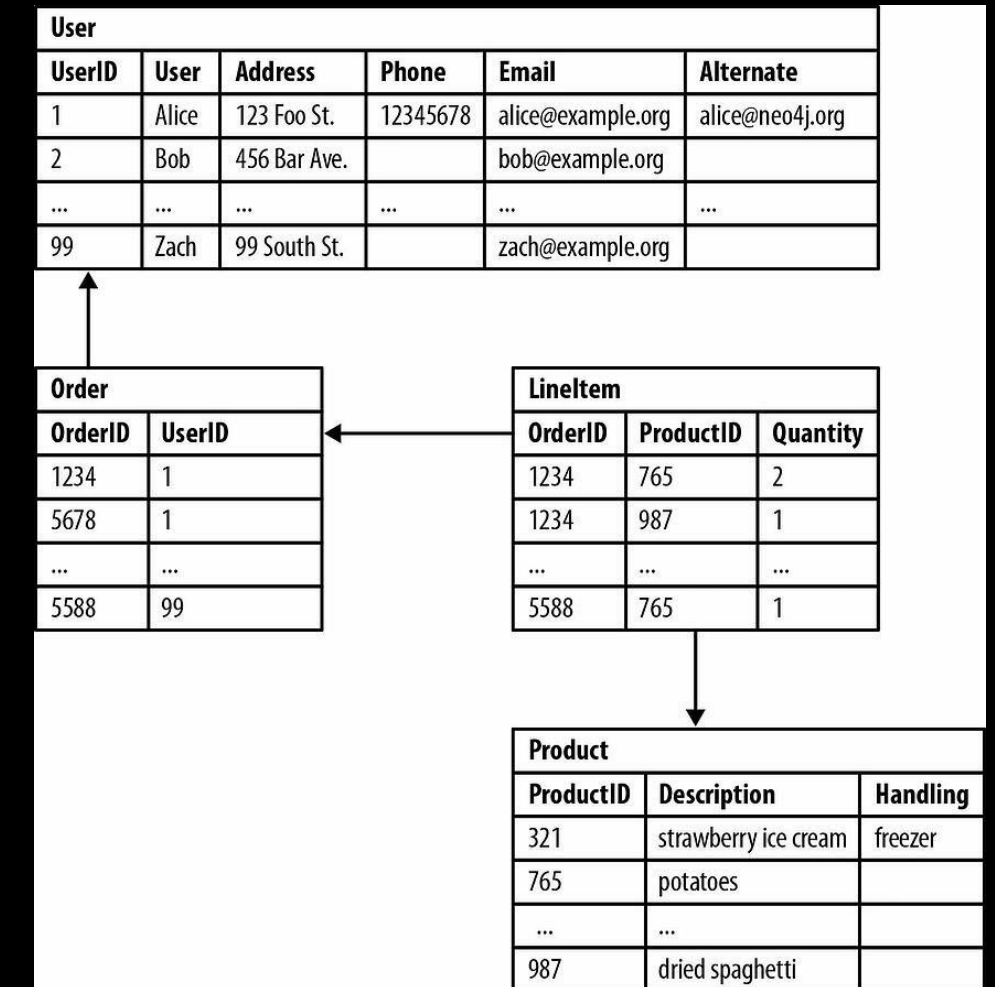
1. Information Retrieval

Information Retrieval 구조도

- RAG의 절차 중 Reranking이 필요한 부분
 - 데이터 벡터화
 - 인덱싱
 - 벡터 DB 저장
 - 쿼리 기반 검색 및 회수
 - 후처리



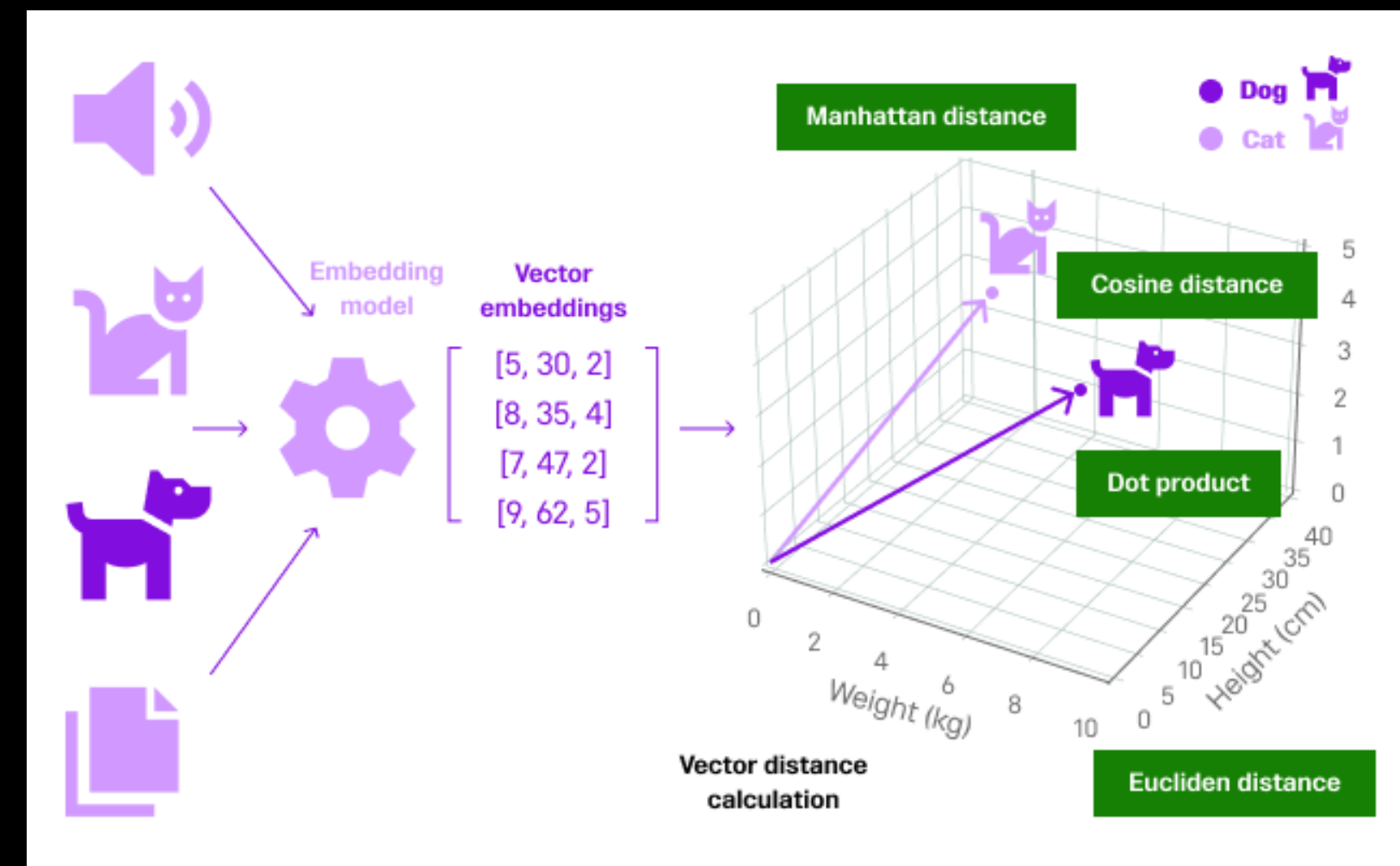
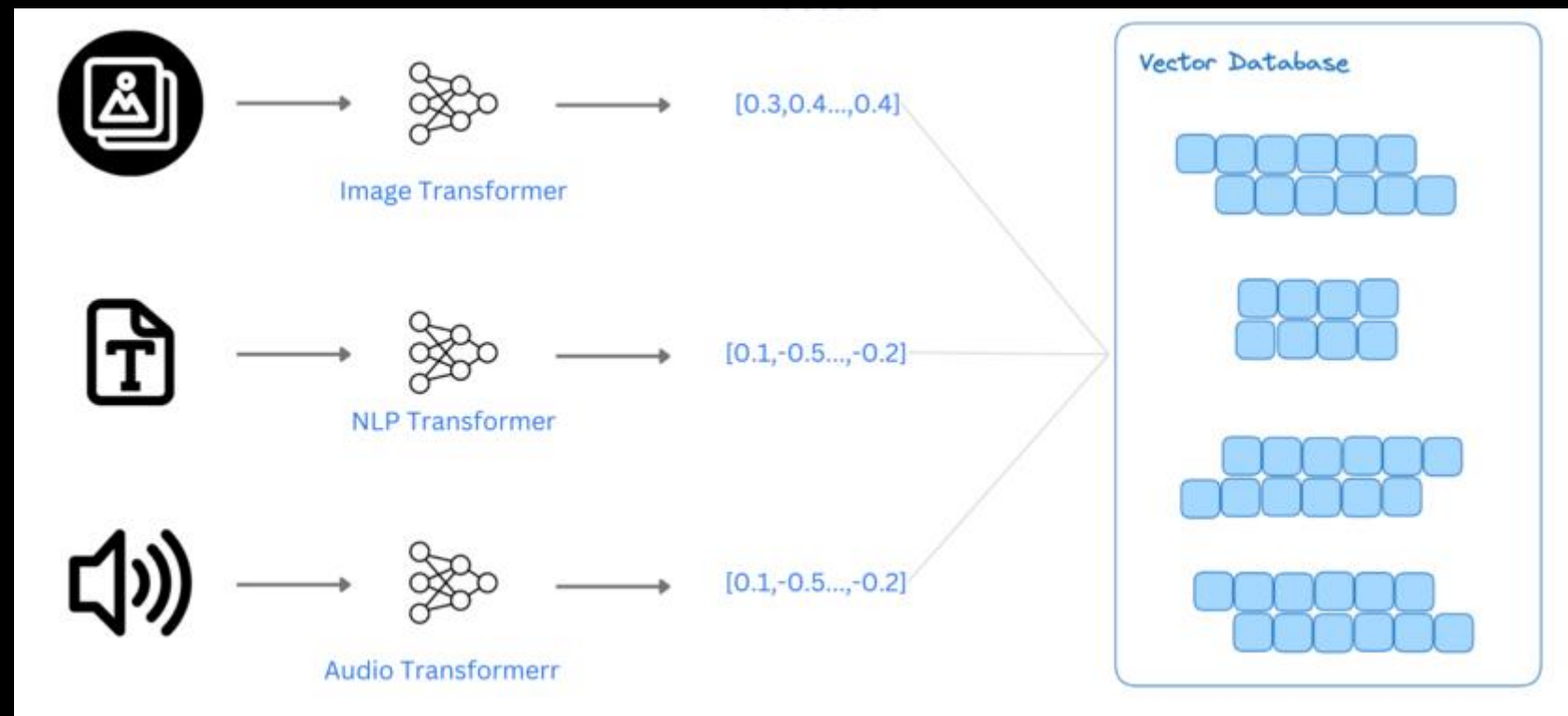
- RDBMS(Relational Data Base Management System)
- 기존 데이터베이스는 테이블 형식으로 데이터를 저장하고 관리
- Row(행) 마다 단일 Record/sample을 표현
- Column(열)마다 데이터의 속성(Attribute)를 나타내며, 수치/문자열 등의 데이터만 기재 가능
- 2차원 형식의 데이터만 기록 가능하므로 제약 사항 많음
- 비정형 데이터베이스 또한 관리가 힘들



데이터 벡터화

• 벡터 DB

- 복잡한 데이터, 특히 텍스트, 이미지, 음성 등의 데이터를 벡터로 변환
 - 하나의 모달리티를 바탕으로 다른 모달리티 검색 가능
 - 텍스트 <-> 이미지 검색
 - 의미상 검색(Semantic search)이 가능
 - 기존 DB에 비해 크기가 작음

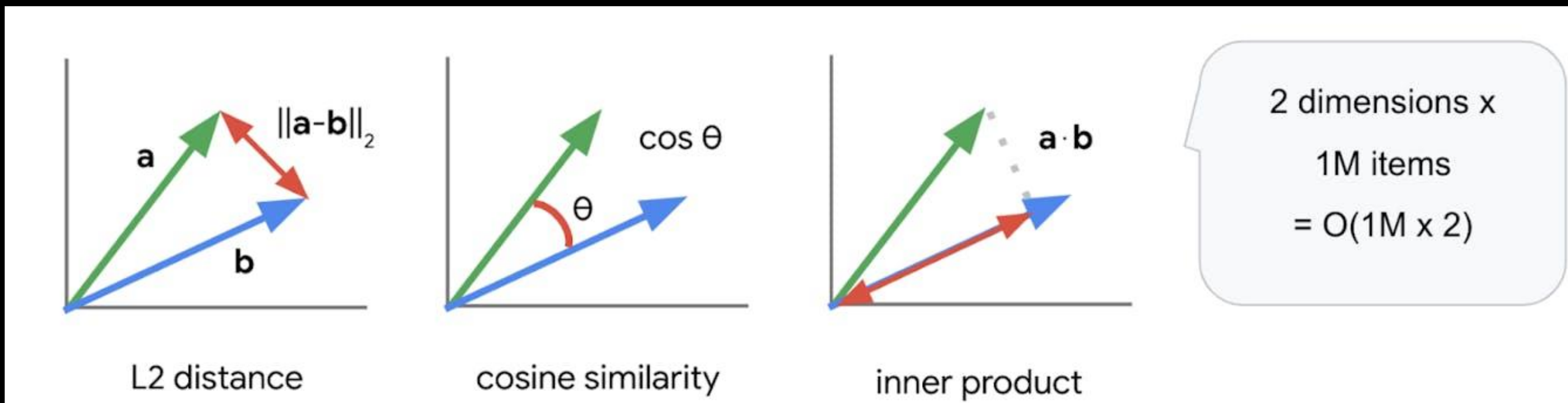




데이터 벡터화

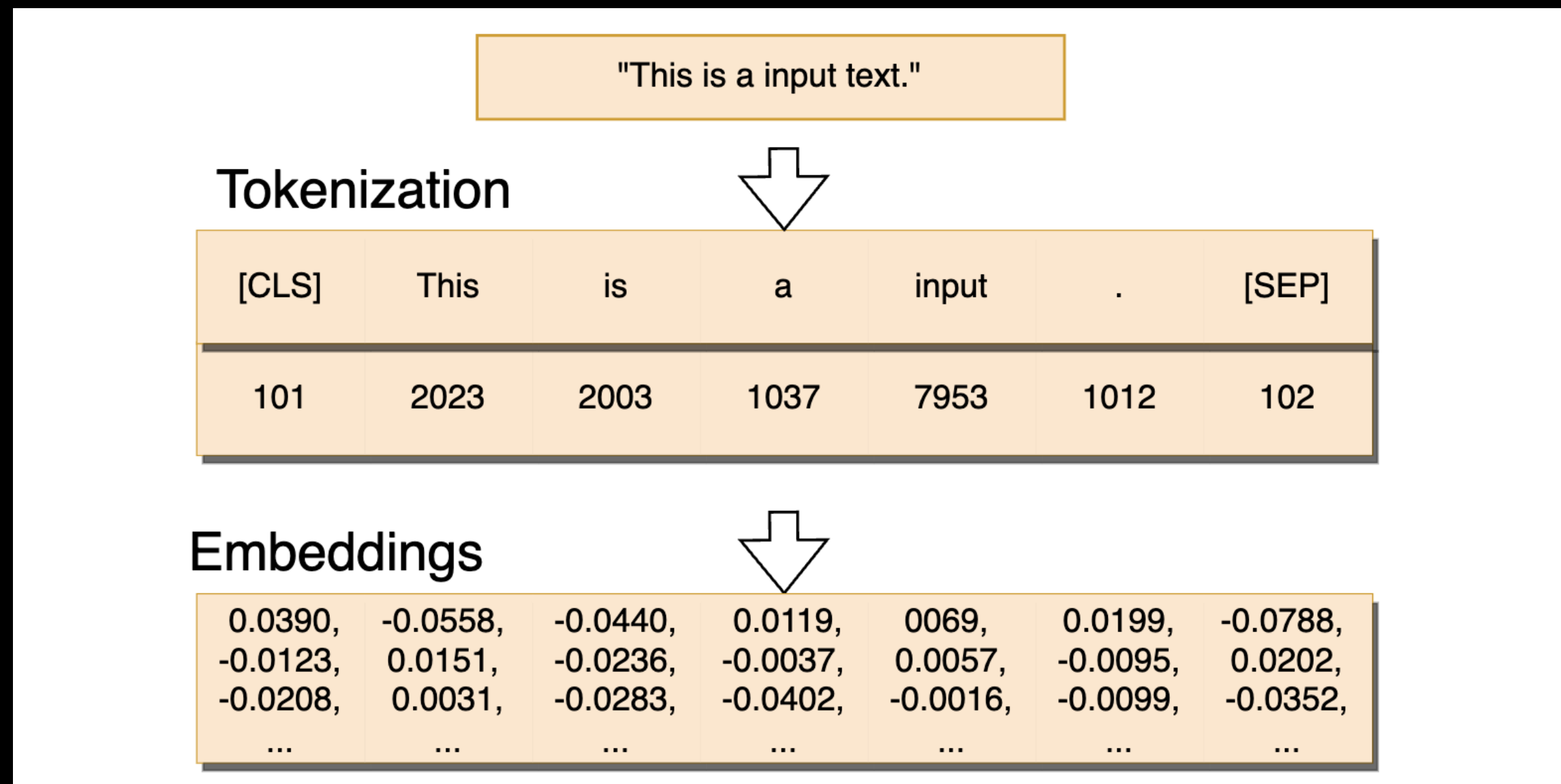
• 벡터 DB

- 데이터를 벡터 형태로 저장하고, 이를 기반으로 유사성을 검색하는 데 최적화된 데이터베이스
 - 원래는 추천 시스템에서 사용
 - 유튜브 추천 알고리즘, OTT 알고리즘 등
- 거리, 각도, 내적 등으로 벡터 간 유사도 계산 가능
 - 벡터 간 내용이 비슷할 수록 거리가 가까움



- 벡터화(Vectorization)

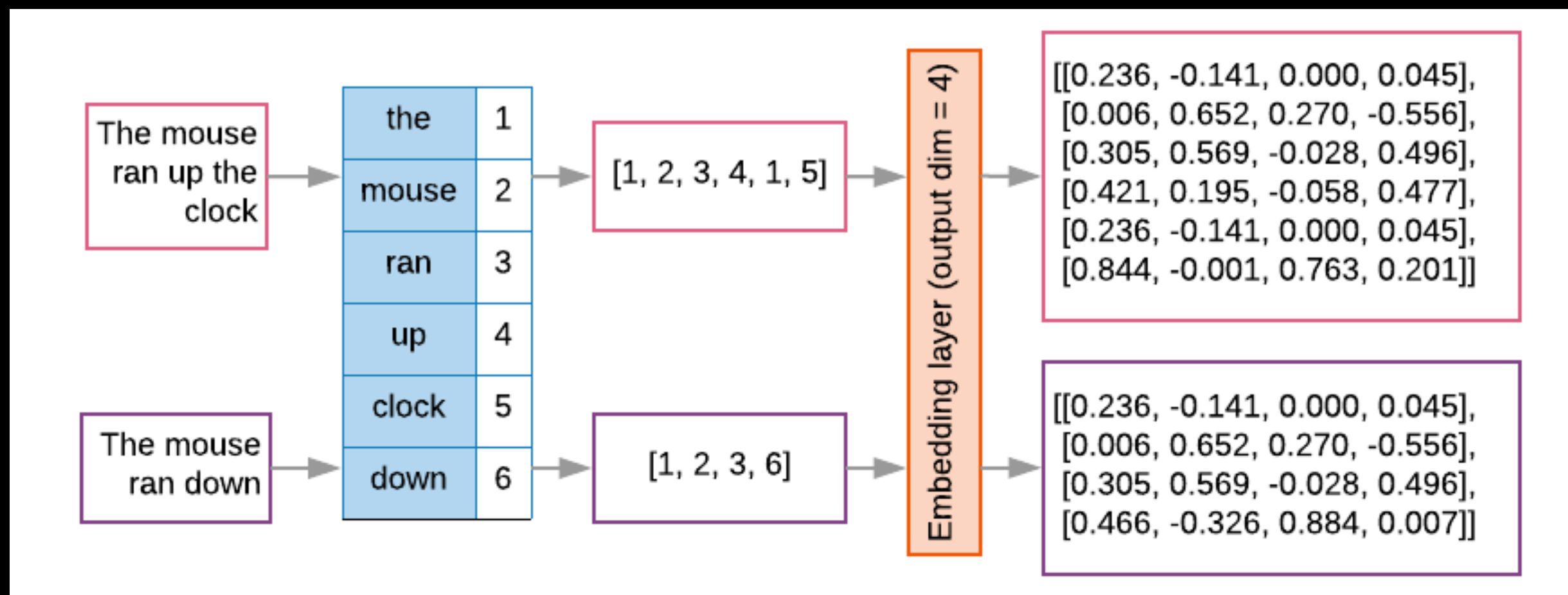
- 데이터를 기계 학습 알고리즘이 이해할 수 있는 형태로 변환하는 것
- 데이터의 형식이 아닌 의미를 수치로 환산하는 과정
- 단어, 문장, 문서 등 다양한 수준의 텍스트 데이터를 벡터로 표현할 수 있음





데이터 벡터화

- Dense vector(밀집 벡터)
 - 정수 혹은 0이 아닌 소수로 표현
 - 데이터의 특징을 표현할 경우, 이들 간의 거리, 연관성 등을 계산하여 유사도를 파악할 수 있음
 - 또한 세밀하고 정교한 의미 표현이 가능하며, 의미 기반 검색(Semantic search)이 가능
 - 신경망을 통해 추출 가능

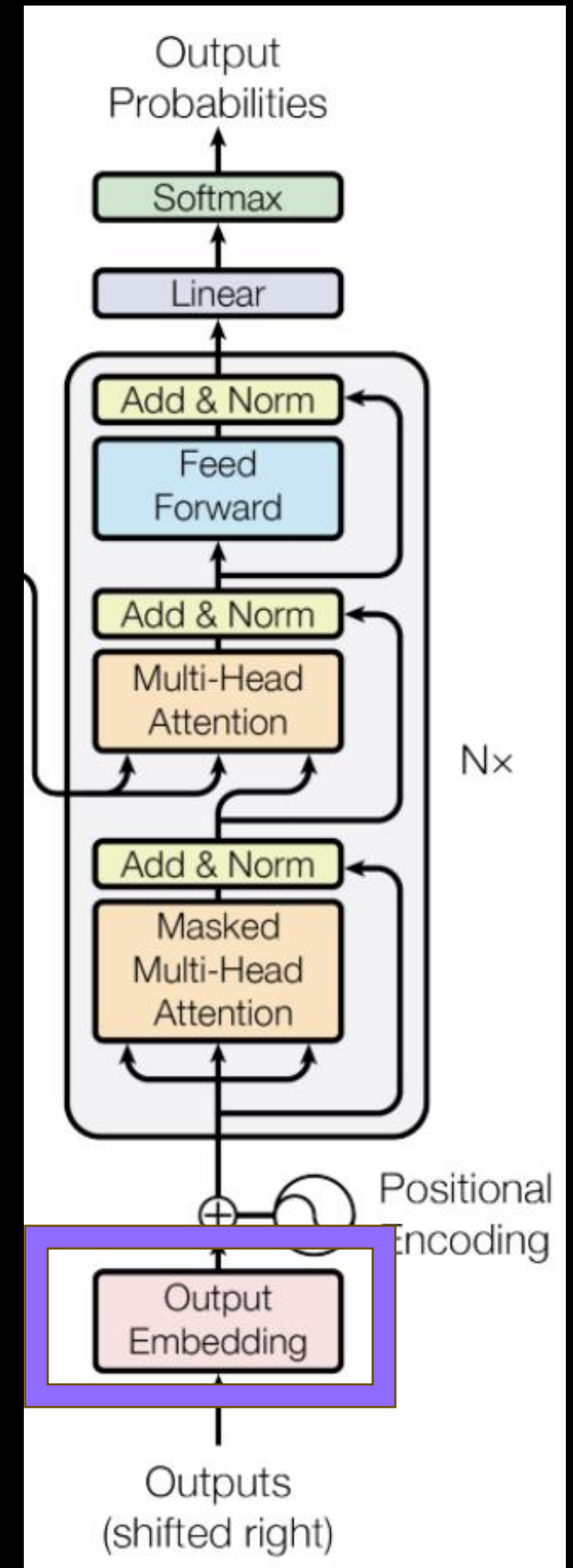




데이터 벡터화

- LLM Embedding

- LLM의 Embedding 성능을 이용하여 단어의 의미를 표현하는 방법
- 대량의 텍스트 데이터로부터 자동으로 학습
 - 수동으로 Feature를 설계하는 것에 비해 더 효과적
- 현재 대부분의 Transformer 기반 모델은 자체 Embedding layer를 갖고 있음
 - API(OpenAI Embeddings, Upstage Solar)
 - Transformer 기반 LLM의 Embedding layer를 추출하여 사용(Pooling)
 - 더 고차원 모델을 사용할 경우 Sentence encoder(SBERT, SElectra 등)





데이터 벡터화

- 희소 벡터(Sparse vector)
 - 몇몇 데이터는 1이고 나머지는 모두 0인 벡터
 - 행/열에 1인 값이 하나만 존재할 경우 One-hot encoding이라 함
 - 특정 단어를 표현할 때, 벡터에서 그 단어에 해당하는 인덱스만 1이고 나머지는 모두 0
 - 단어장(Vocabulary)의 크기를 벡터의 차원으로 하며, 각 단어는 단어장에서의 위치(Index)에 따라 하나의 차원에 1의 값을 가짐
- 신경망이 등장하기 전에는 빈도 기반으로 임베딩
 - 이 때 희소 벡터로 단어의 중요도를 평가

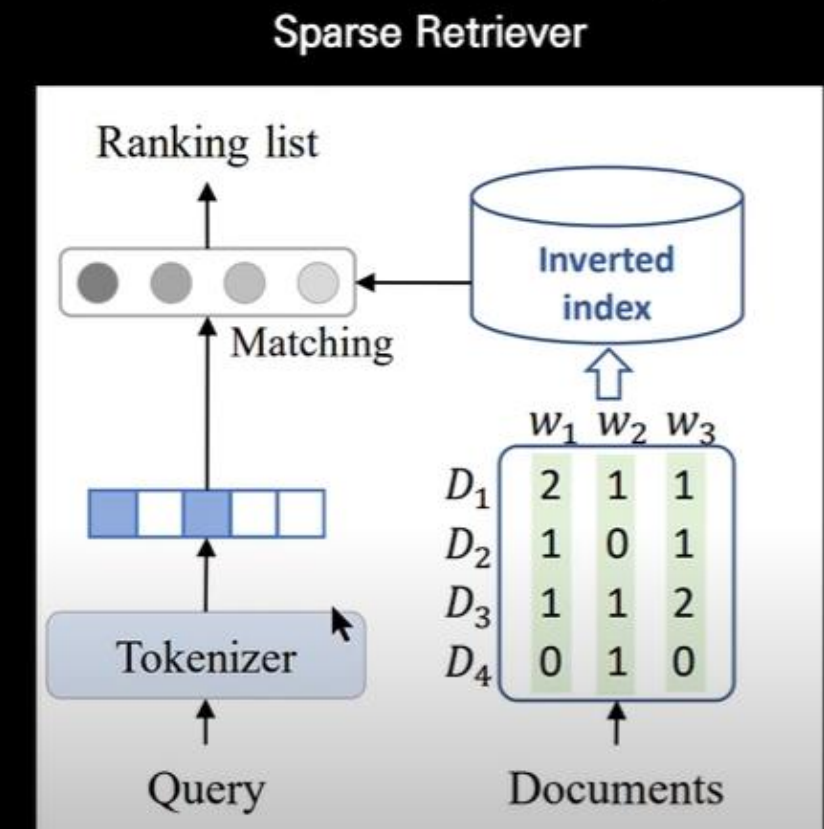
	cat	mat	on	sat	the
the =>	0	0	0	0	1
cat =>	1	0	0	0	0
sat =>	0	0	0	1	0



데이터 벡터화

- 구현이 쉽고 이해하기 직관적
 - 키워드 기반 검색 가능
- 그러나 세 가지 큰 문제가 발생
 - 차원의 저주: 단어장의 크기가 클 경우 벡터의 차원이 지나치게 커지며 연산에 불리한 구조를 갖게 됨
 - 대부분의 값이 0으로 표현되기에 연산의 결과에 0이 많이 포함
 - 단어 간 유사성을 전혀 표현하지 못함
- 위와 같은 이유로 LLM Embedding에서는 배제되었으나, 정보 검색(IR) 분야에서 재활용
 - 대표적인 빈도 기반 Embedding 방법: TF-IDF

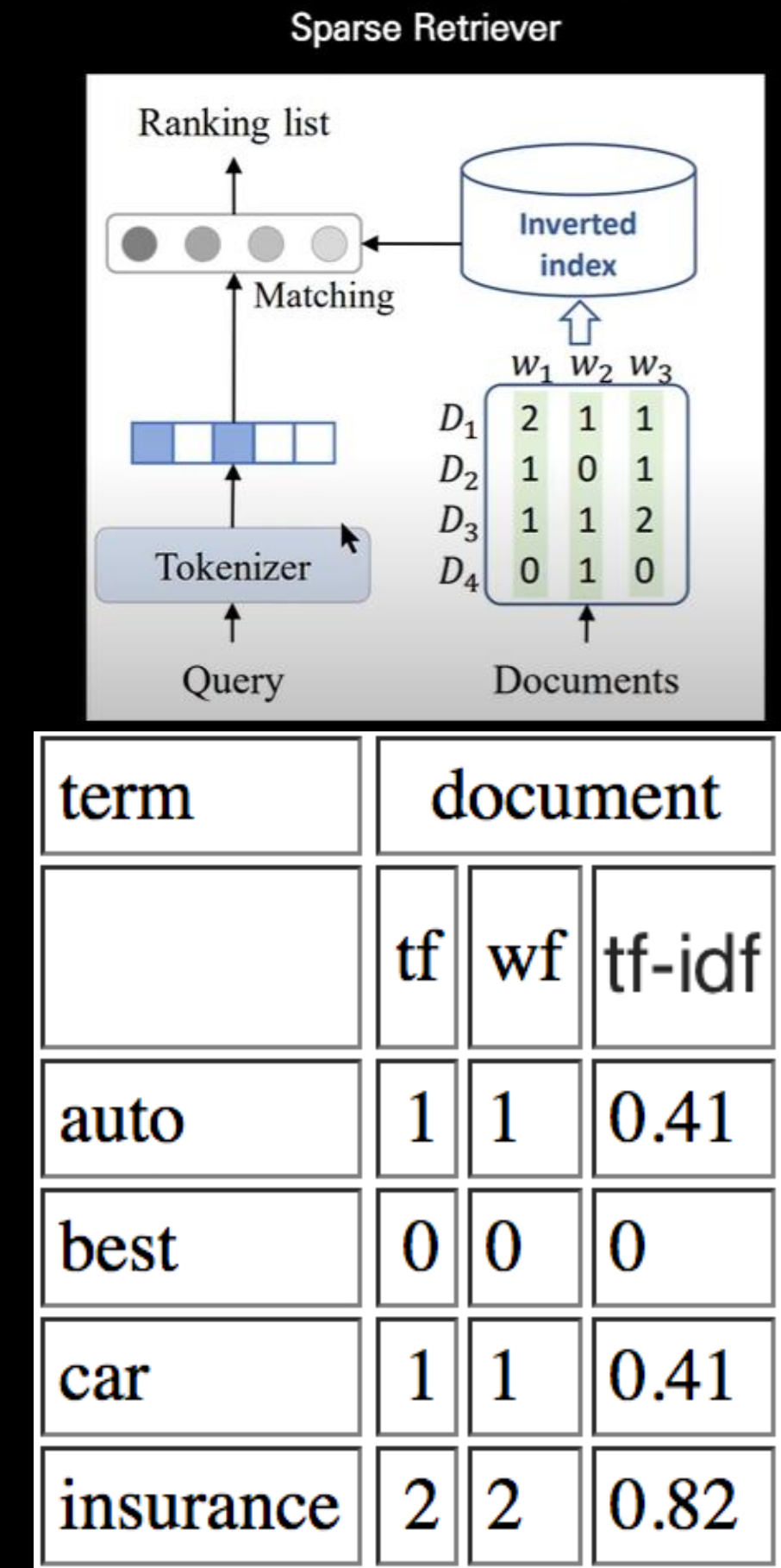
- TF-IDF(Term Frequency-Inverse Document Frequency)
- TF(Term Frequency)
 - 특정 단어가 하나의 문서 내에서 등장하는 빈도
 - 이는 단어가 문서 내에서 얼마나 중요한지를 나타내는 지표



term	document		
	tf	wf	tf-idf
auto	1	1	0.41
best	0	0	0
car	1	1	0.41
insurance	2	2	0.82

데이터 벡터화

- TF-IDF(Term Frequency-Inverse Document Frequency)
- IDF(Inverse Document Frequency)
 - 특정 단어가 등장하는 문서의 수에 반비례하는 수치
 - 모든 문서에서 자주 등장하는 단어는 중요도가 낮다고 판단
 - 특정 문서에서만 자주 등장하는 단어는 중요도가 높다고 판단
- TF-IDF:
 - $TF * IDF$
 - 값이 높을수록 단어의 중요도가 높다고 판단
 - 단어의 빈도와 문서 내에서의 중요도를 동시에 반영하게 됨
 - VectorDB의 텍스트와 쿼리 간 문서 내 단어 빈도를 바탕으로 탐색

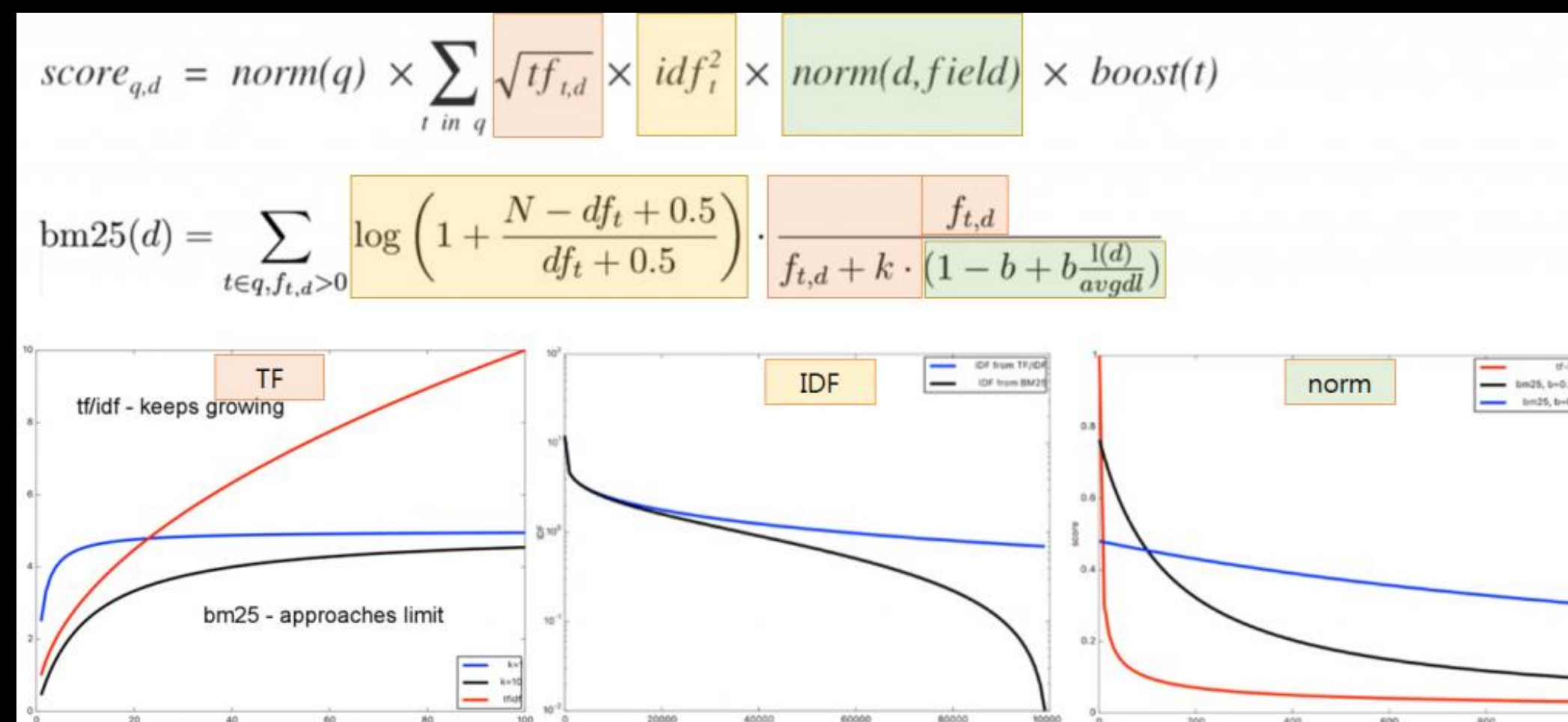
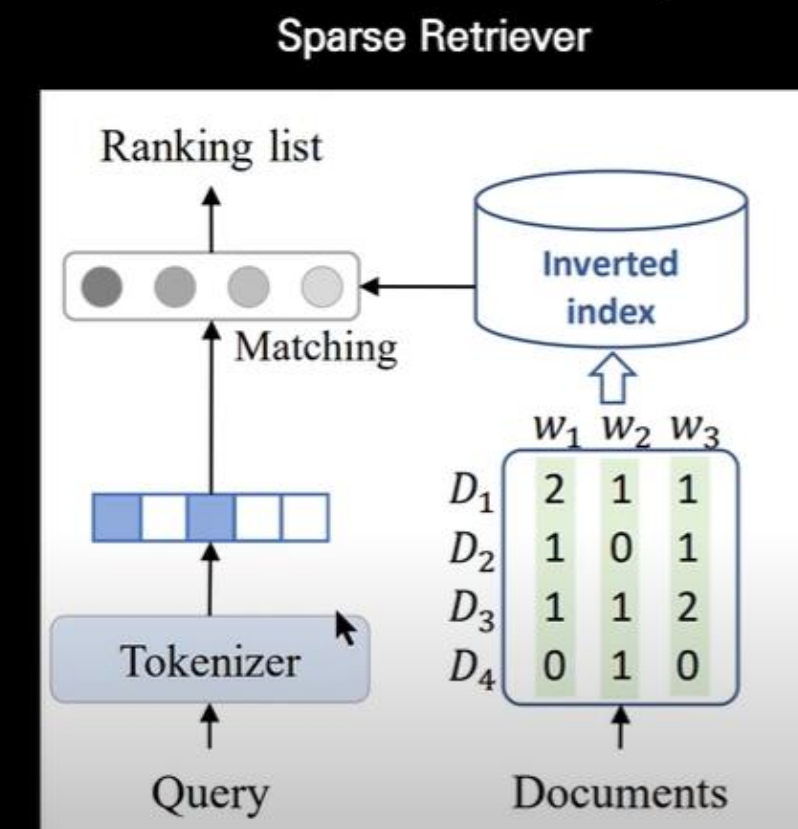




데이터 벡터화

• BM25

- TF-IDF에 보완점을 추가한 IR 계산 알고리즘
- 단어의 등장 빈도(TF)가 많이 등장하더라도 관련 점수가 과도하게 증가하지 않음
 - Saturation function을 적용
- 희소한 단어(IDF)에 높은 점수를 부여
- 길이 정규화
 - 문서가 길 수록 단어를 더 많이 포함
 - 길이가 상이한 문서 내 점수를 조정

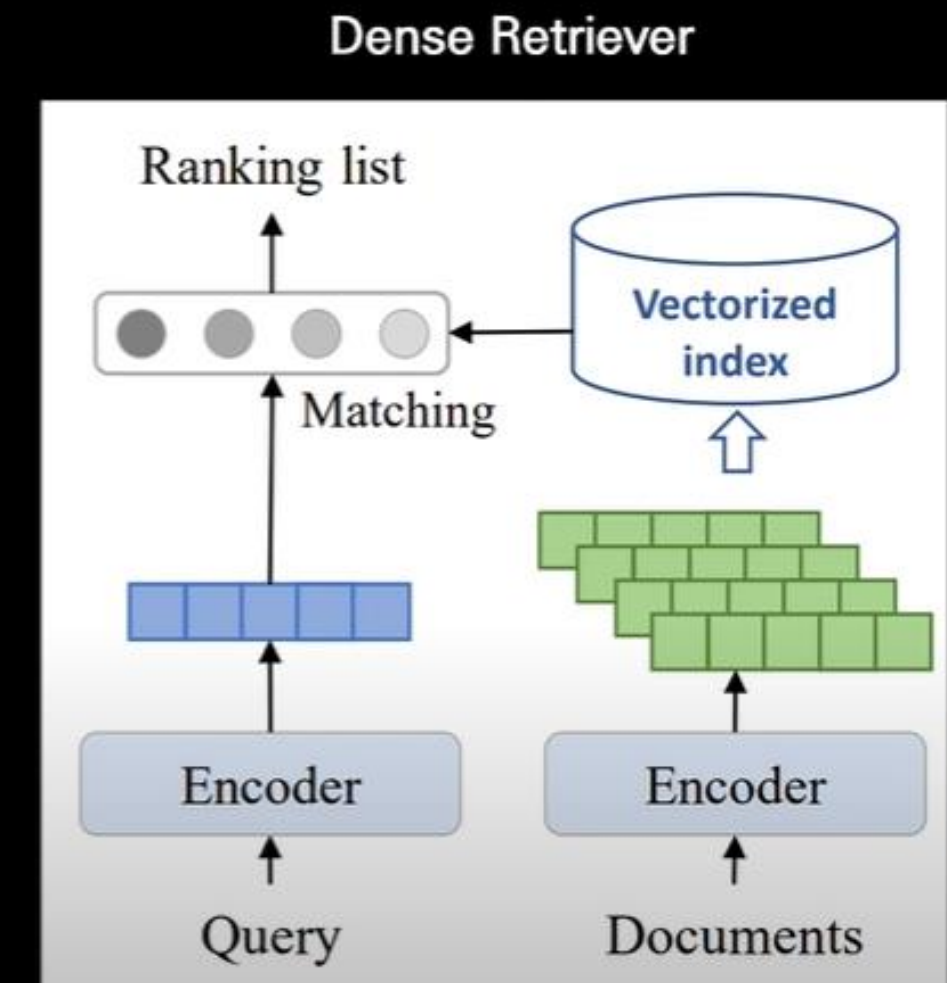


- Sparse retriever

- 문서/쿼리에서 등장하는 단어나 키워드를 기반으로 검색하는 방법
- Embedding이 의미 기반이 아닌 빈도 기반으로 수행됨
 - 별도의 신경망을 거치지 않으므로 속도 측면에서 이점이 큼
 - 정확도가 떨어짐

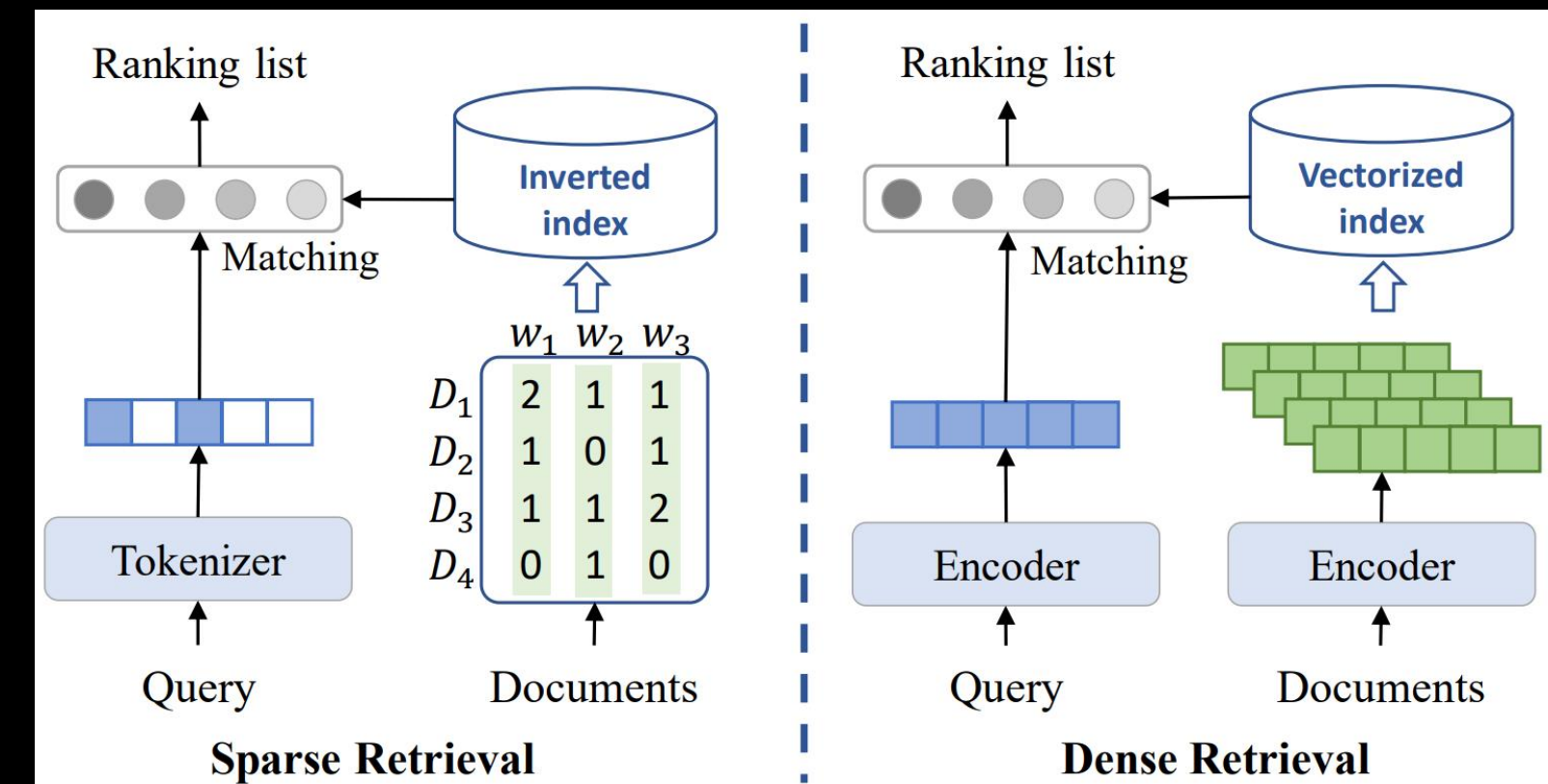
- Dense retriever

- DL 모델을 통해 학습된 임베딩을 활용하여 검색하는 방법
- 또는 BERT, RoBERTa 등의 모델로 질의와 문서의 의미를 벡터 공간에 매핑하여 유사도를 측정
- 키워드가 일치하지 않더라도 의미상 동일한 문서를 검색할 수 있음(Semantic search)



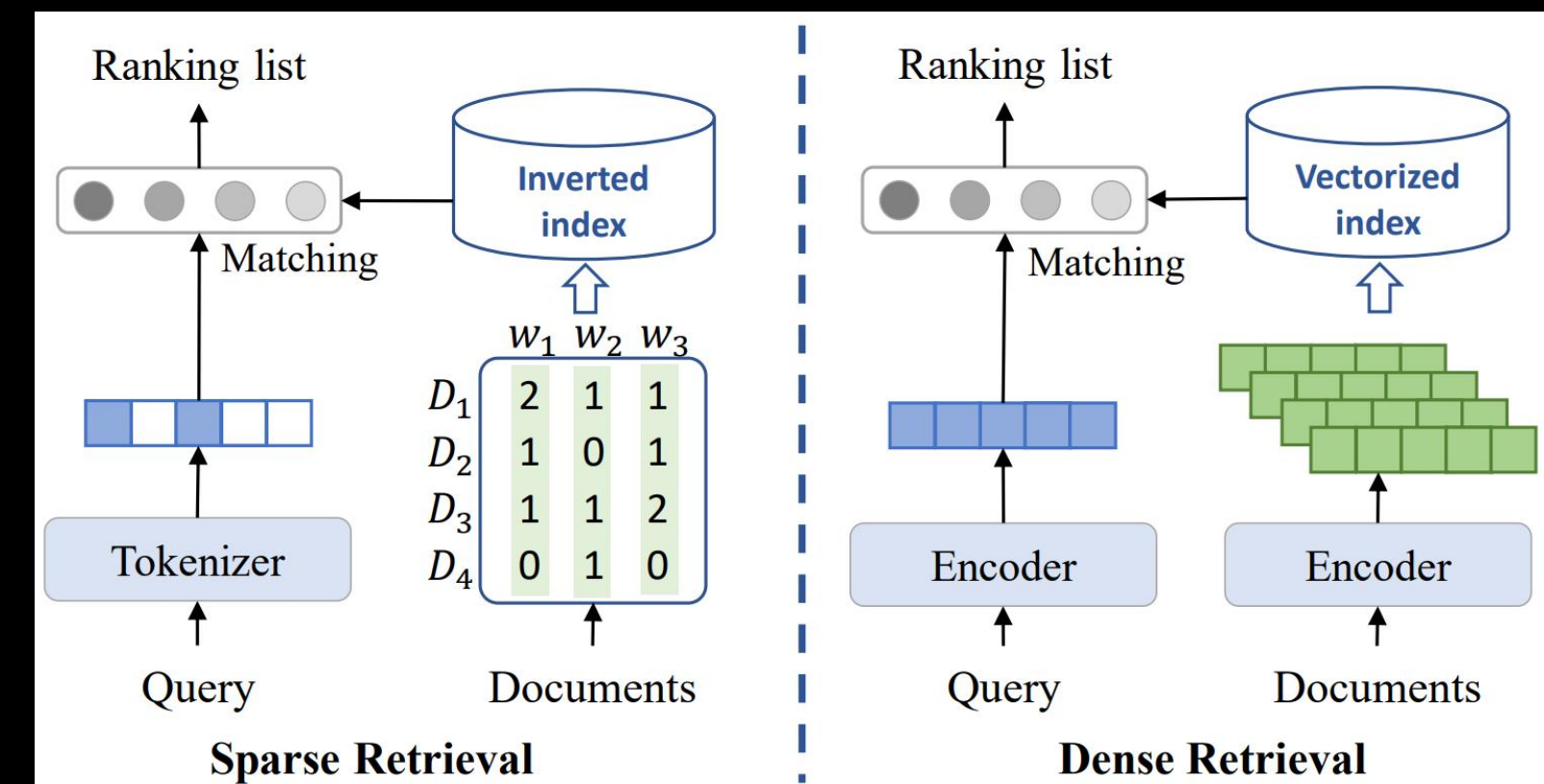
Hybrid Search

- Sparse retrieval과 Dense retrieval을 결합하여 인덱싱
 - Sparse retrieval
 - 키워드 기반 검색을 통한 핵심 단어 일치 여부 능력
 - 키워드가 포함되지 않는 경우 존재
 - Dense retrieval
 - 의미 기반 검색(Semantic search)을 통하여, 매뉴얼 검색의 단점 보완
 - 다만 전문성 높은 문서에서는 정밀한 의미 구분이 어려울 수 있음
 - E.g) "2024년에 3월 매출 실적에 대하여 요약해줘"
 - 2024년이 아닌 때의 매출 관련 문서
 - 2024년 3월의 매매 관련 문서



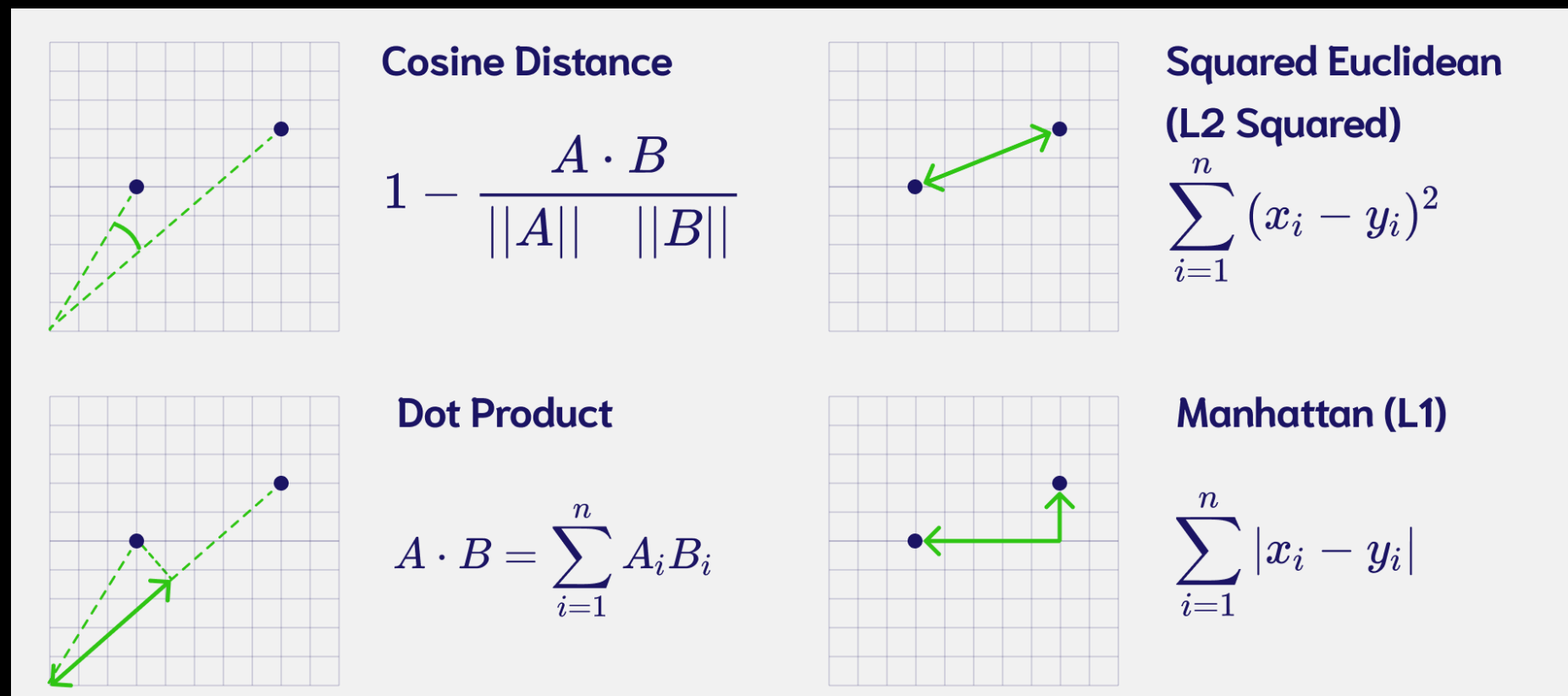
Hybrid Search

- Hybrid search를 구성하기 위해선
 - 동일한 Chunk를 서로 다른 두 벡터 DB로 구성하거나
 - 한 벡터 DB 내 동일 Chunk에 대하여 두 가지 key값(Sparse vector, Dense vector)가 필요
 - 서로 다른 두 방법을 통해 Query와 유사한 Chunk를 찾을 경우 각기 다른 Context가 회수됨
 - 그러나 이들 중 Query에 대응하는 중요한 것을 선별하는 과정이 필요



- RAG는 검색 기반 응답 시스템
 - 내용이 정확해야 하며
 - 검색속도 또한 빨라야 함(<10s)
- 그러나 RAG에는 속도가 느려질 수 있는 요인이 매우 많음
 - LLM에서 응답을 생성하는 과정에서 시간이 길게 소요
 - 검색 결과의 정확도를 높이기 위하여 Vector DB의 크기를 키울 수록 속도는 느려짐
 - 속도를 개선하기 위해선 정보 탐색 과정의 변화가 필요

- Vector-level indexing
 - Dense vector 기준 다양한 방법을 통해 Chunk 간 유사도 계산 가능
 - 각도, 거리, 내적, MMR(Maximum Marginal Relevance) 등
 - 그러나 이들 중 절대적인 것은 없음
 - 경우에 따라서 이들을 모두 활용하여 결과를 검색하고, 종합 점수를 바탕으로 회수할 수도 있음
 - 이 경우에도 회수된 Chunk들과 Query의 관계를 재정립할 필요 있음



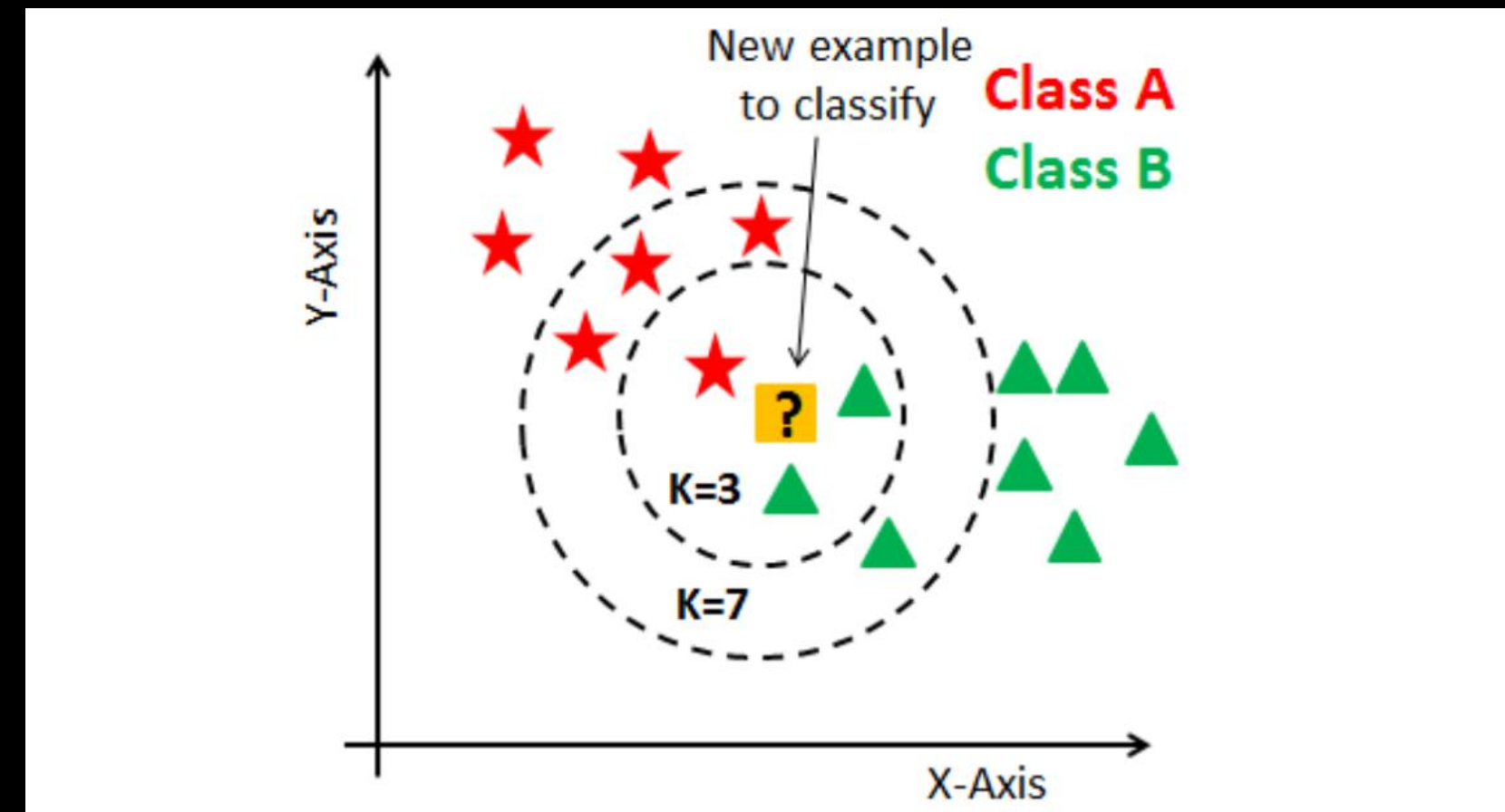
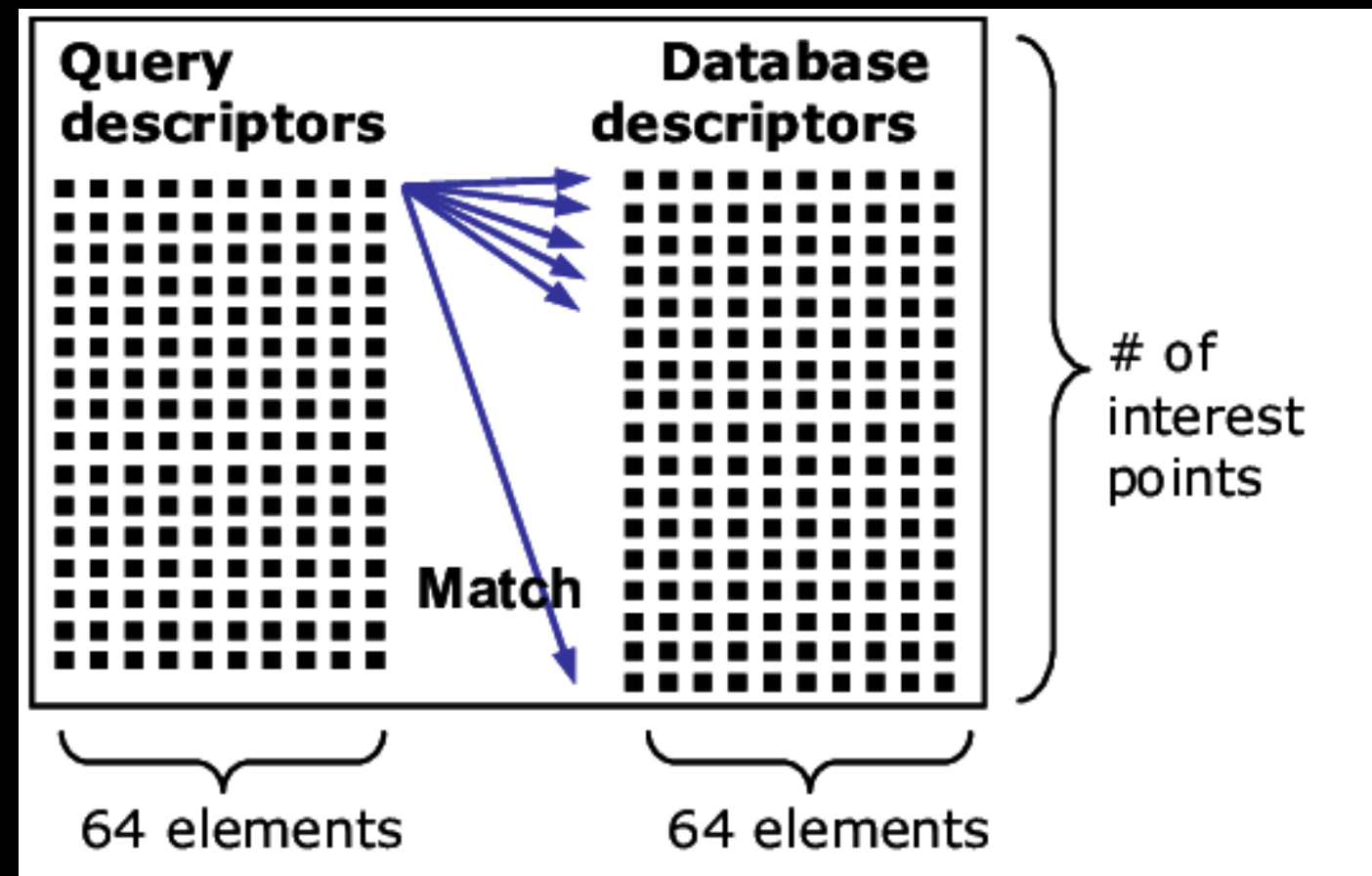


Indexing

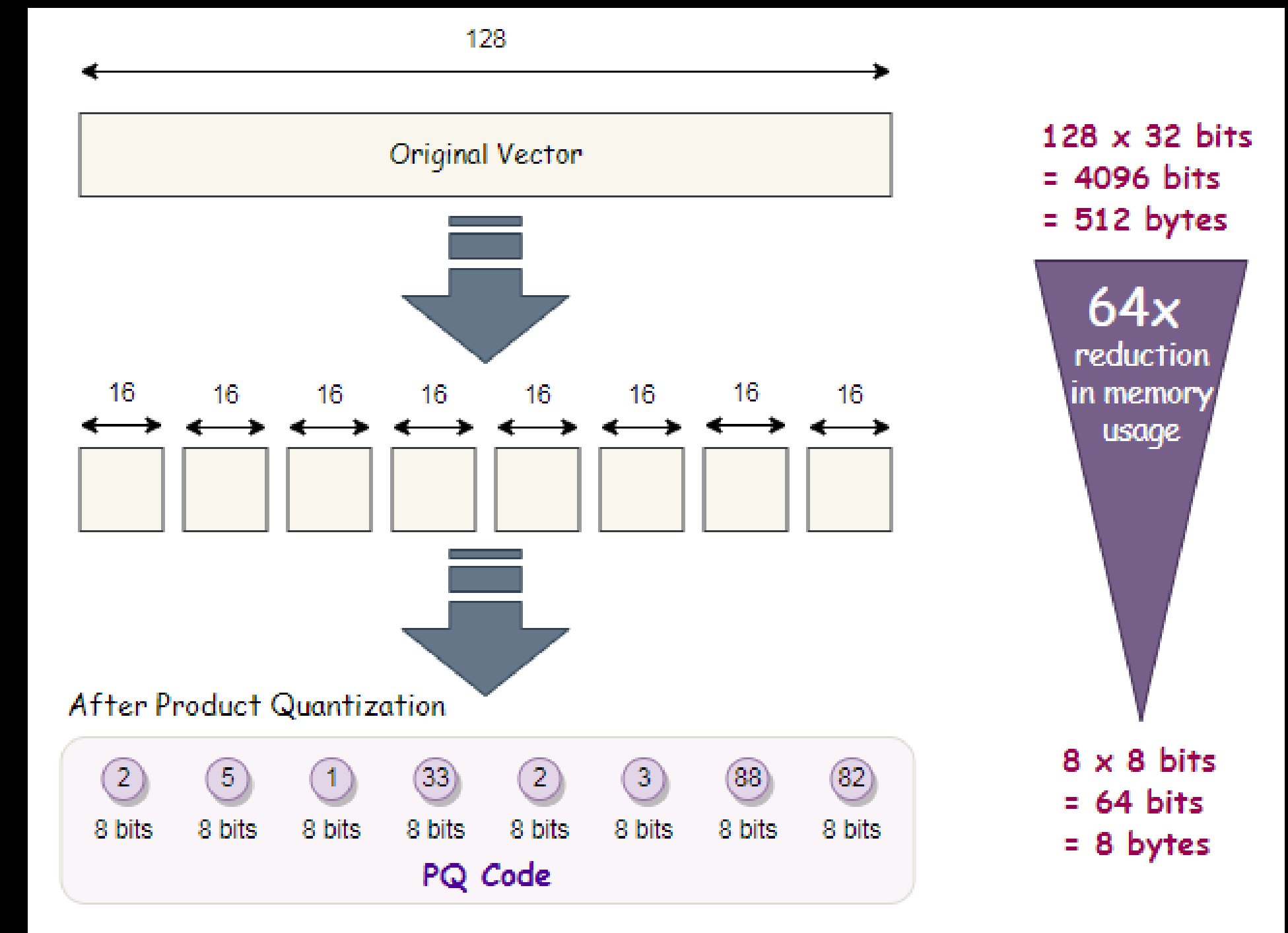
- 검색(Indexing) 속도 최적화 알고리즘
 - Flat index
 - Random projection
 - PQ(Product Quantization)
 - HNSW(Hierarchical Navigable Small World graph)
 - LSH(Locality-Sensitive Hashing)
 - IVF(Inverted File Index)

Indexing

- Flat index
 - Brute-force
 - 쿼리와 각 데이터 사이의 거리를 모두 계산하여 가장 가까운 데이터를 찾음
 - 거리 계산은 Euclidean distance, KNN, Cosine similarity 등을 사용
 - 데이터가 1-5만 여개 까지 잘 작동하며, 정확도가 높음

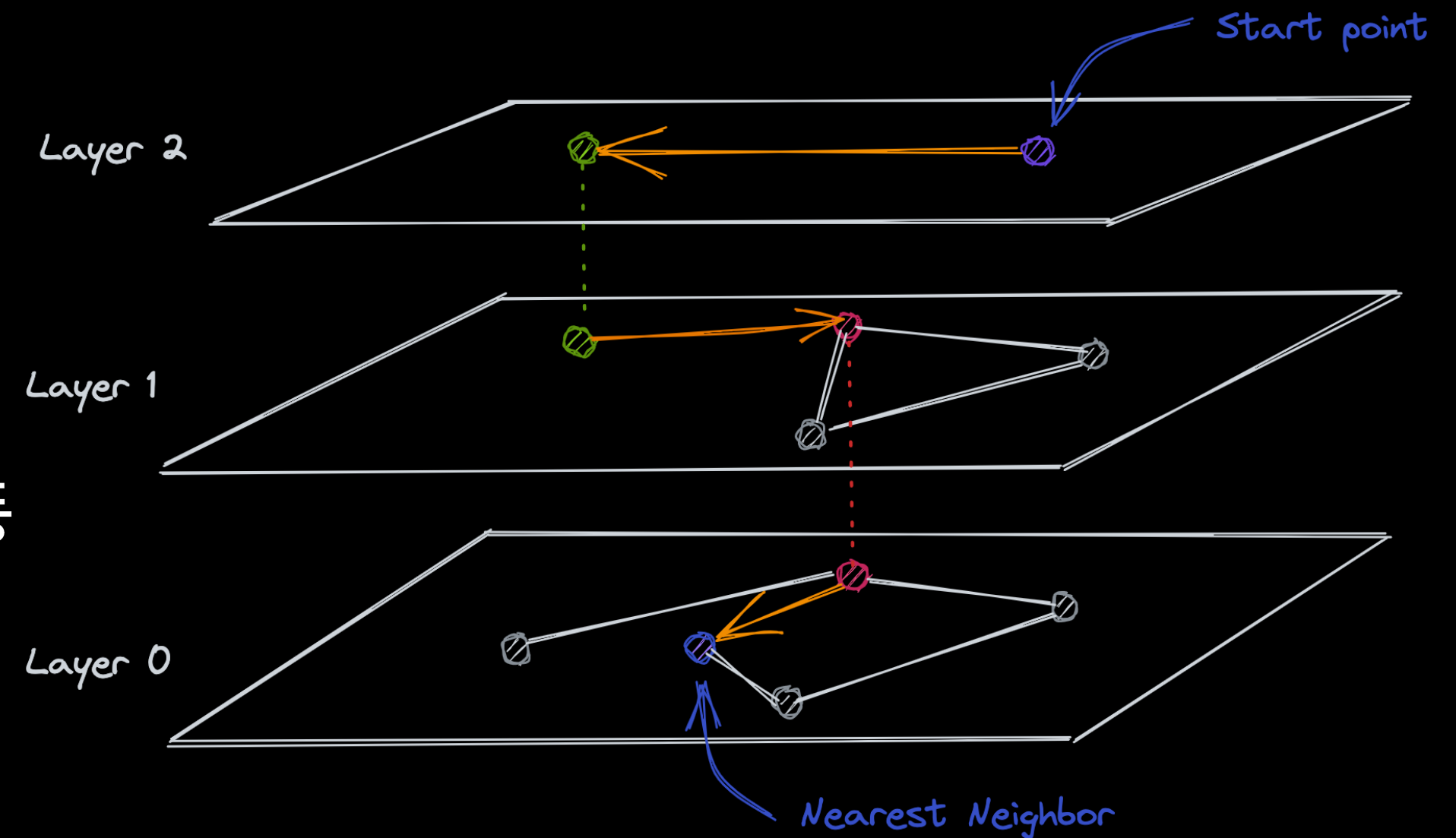


- Product Quantizer (PQ)
 - 벡터를 동일한 크기의 서브벡터 N개로 분할 후
 - 각 서브벡터를 양자화하여 저장
 - 동일한 데이터셋을 더 작은 크기로 저장 가능
 - 속도, 메모리 사용량을 현저히 줄일 수 있음

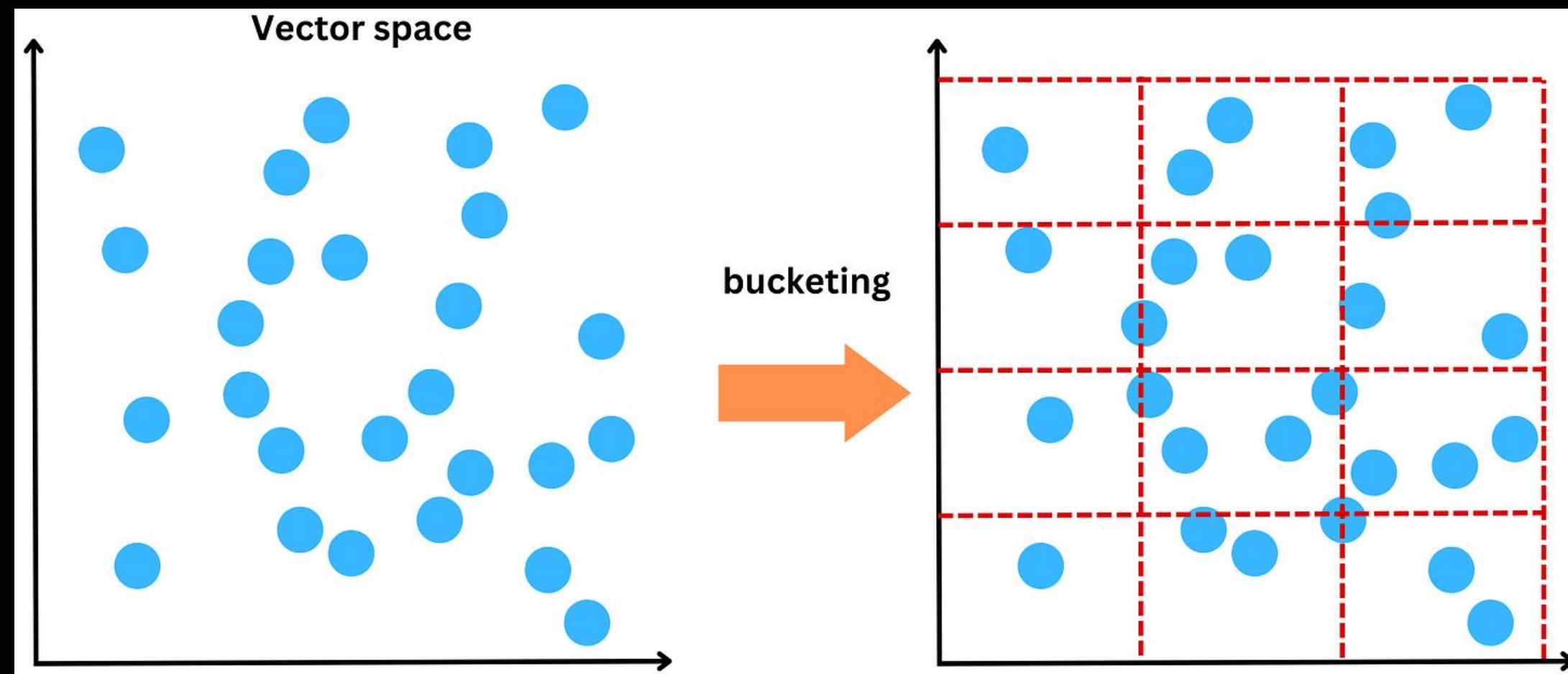


- HNSW(Hierarchical Navigable Small World graph)

- 벡터들을 바탕으로 N개 층으로 구성된 그래프를 생성
 - 층에 따라 포함된 노드의 수가 상이
 - 최하단층에는 모든 노드가 포함
- 노드 간 거리가 벡터 간 유사도
- 최상단층에서 시작하여 탐색을 시작
- 동일 층에서 거리를 더 좁힐 수 없다면 하층으로 이동
- 쿼리 벡터와 가장 가까운 노드 발견 시까지 반복

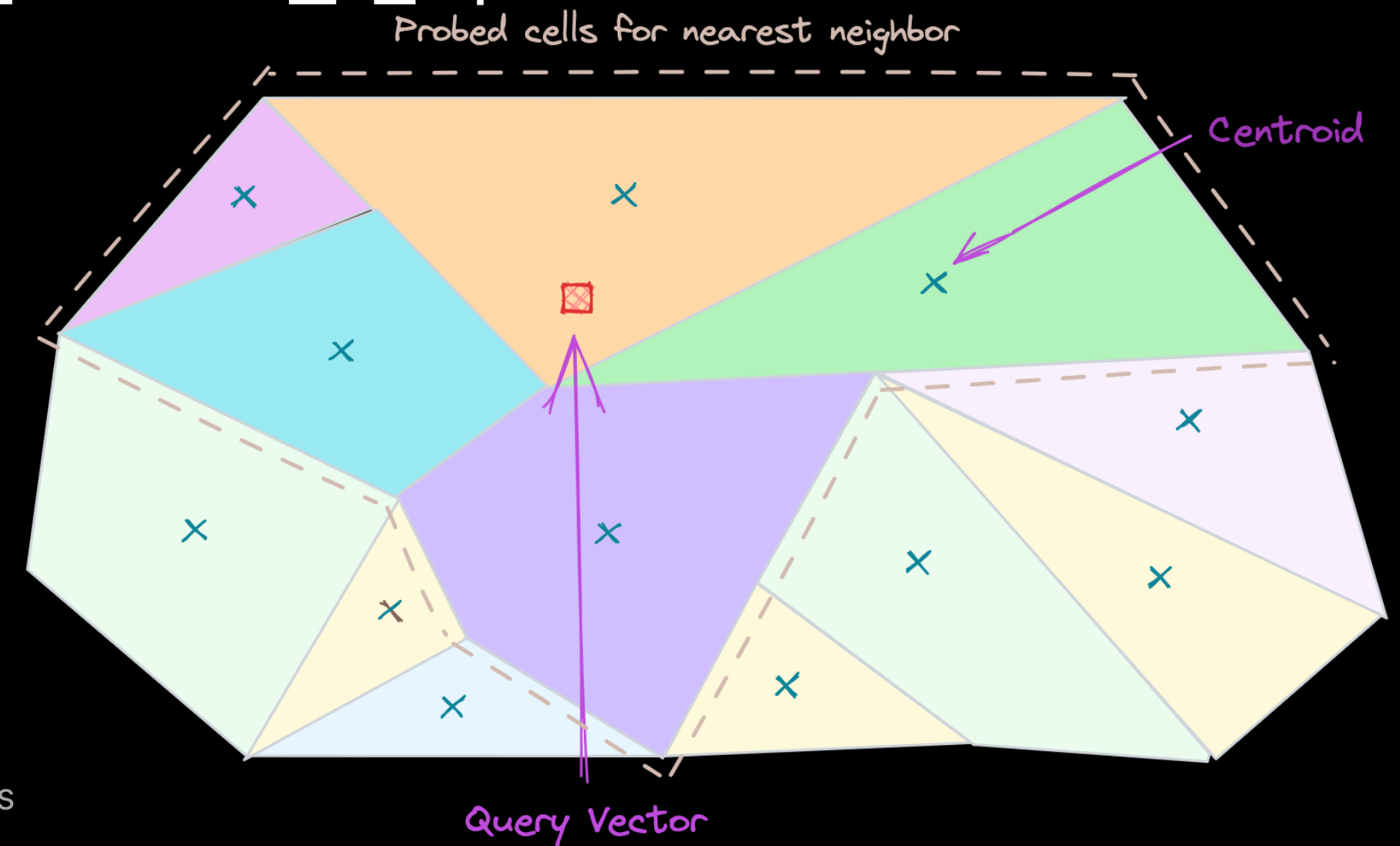


- LSH(Locality-Sensitive Hashing)
 - 데이터를 Hash 함수에 입력하여 서로 다른 Bucket에 mapping
 - 유사도가 높은 원소들을 같은 Bucket에 포함시킴
 - 쿼리를 바탕으로 유사한 벡터 검색 시, 같은 Bucket에 속한 벡터들에 대해서만 비교
 - Bucket의 수가 많을 수록 결과가 좋아지지만, 메모리 소비량 또한 증가

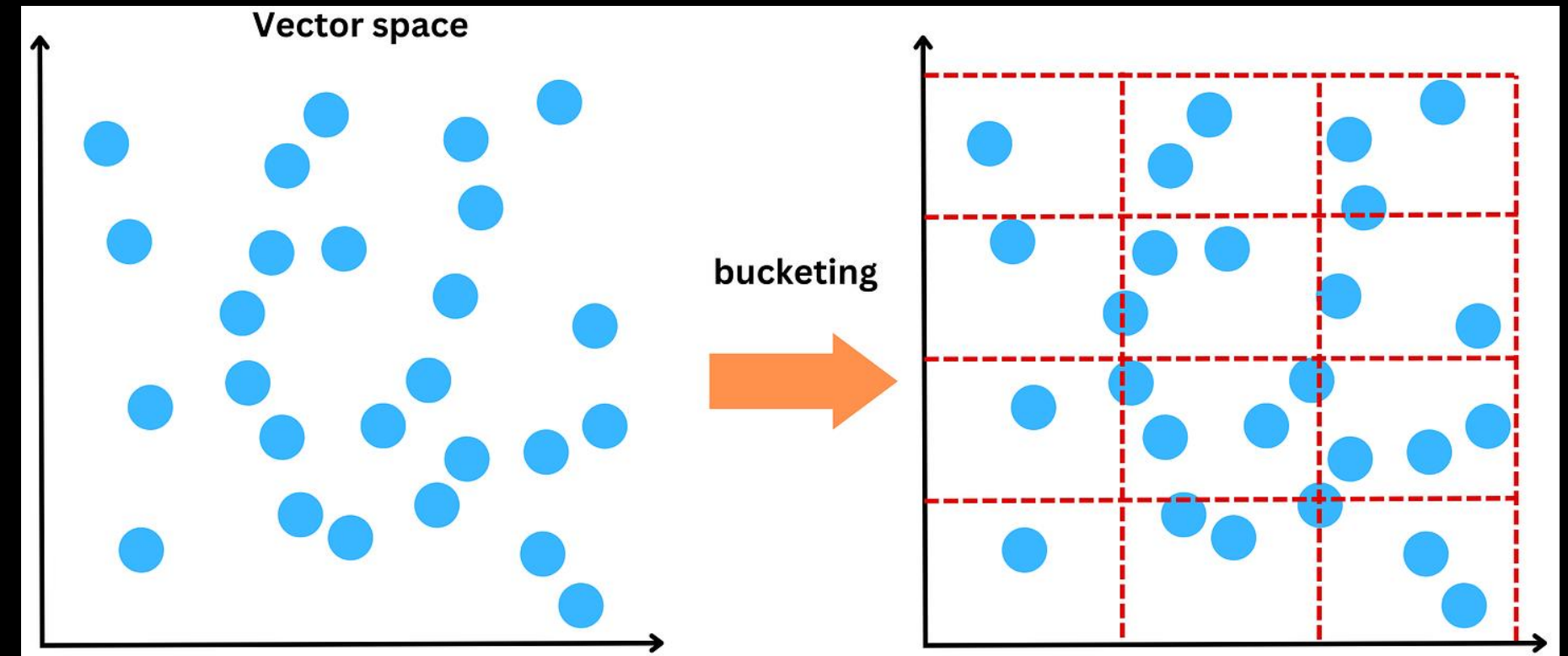
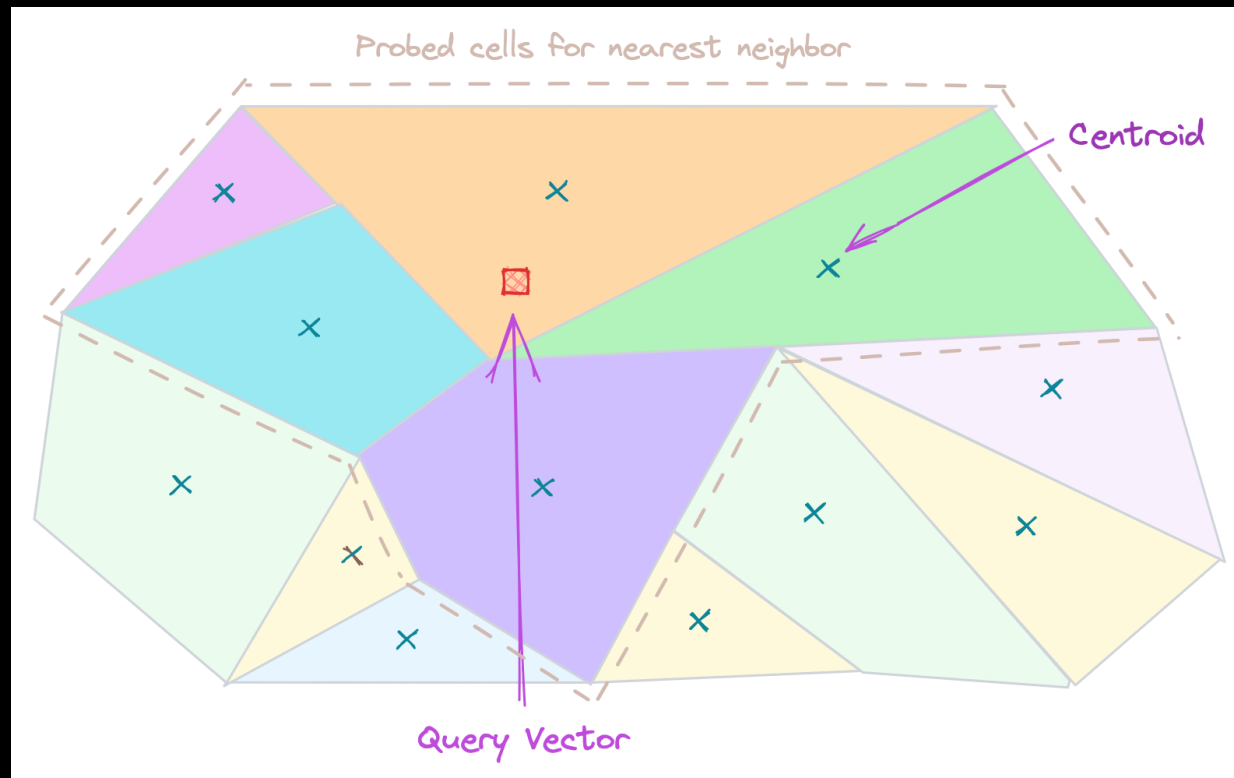


- IVF(Inverted File Index)

- Clustering을 통해 벡터들을 N개의 군집으로 분할
- 각 군집의 중심점(Centroid)를 계산
- 각 벡터의 Index ID와 소속된 Cluster, 중심점 좌표를 리스트에 저장
- 쿼리가 입력될 경우, 이 쿼리의 위치를 바탕으로 소속될 Cluster를 탐색
- 해당 Cluster 안의 벡터들에 대해서만 유사도 검색



- IVF, LSH 방식의 경우
 - 반드시 하나의 Cluster/bucket에서 검색할 필요는 없음
 - Query에 대한 실질적 답변이 모델이 유추한 Cluster/bucket에 존재한다는 보장이 없기 때문
 - 상위 N 개의 Cluster/bucket에서 K 개의 Chunk를 회수할 수도 있음
 - 이 경우 여러 개의 Chunk가 회수될 때 Query와 이들 간의 관계를 재정립하는 과정이 필요

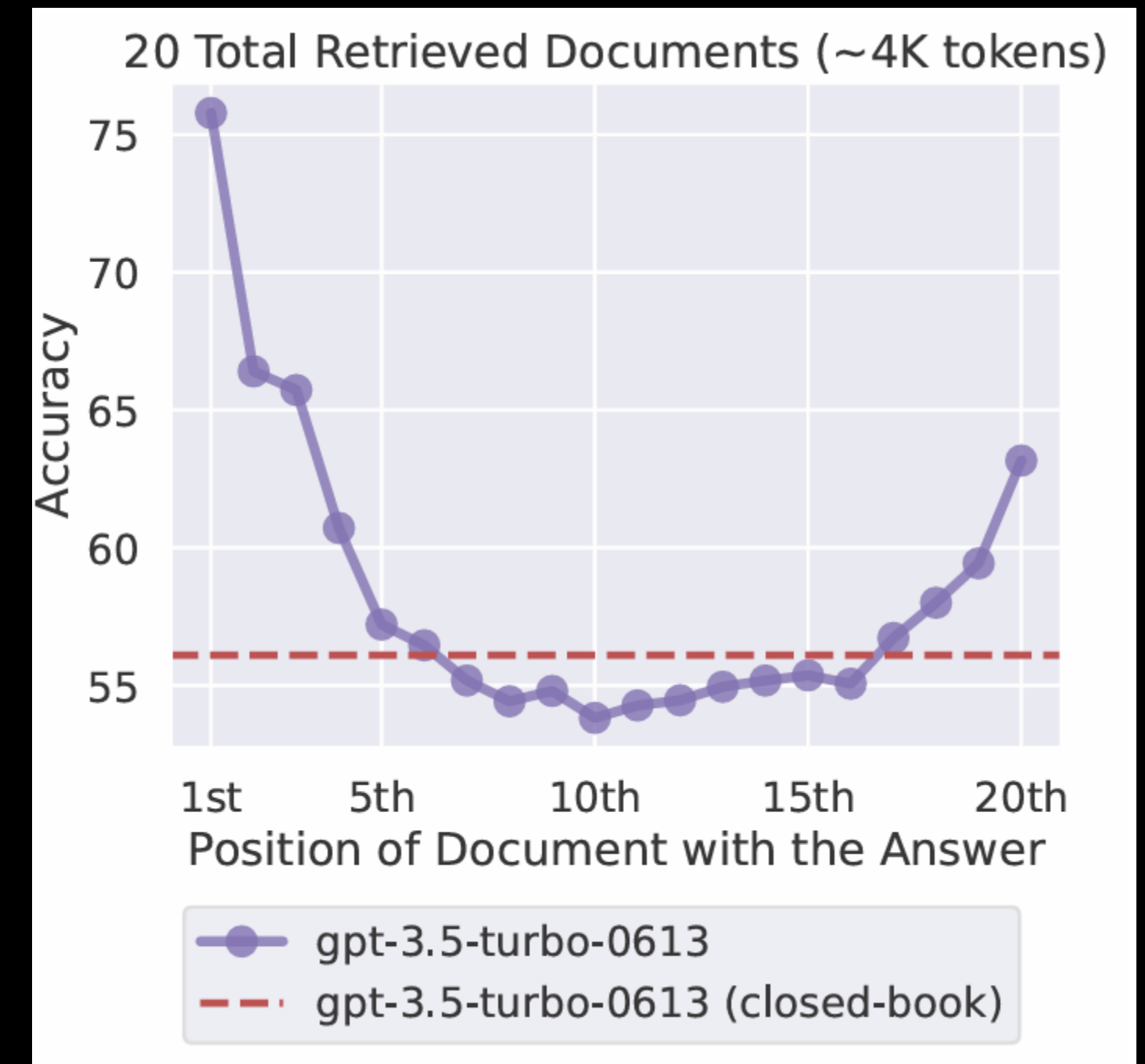


2. Rerank

- 정보검색(IR) 시스템에서 2단계 랭킹을 수행하는 기법
 - 초기 검색 단계에서 비교적 빠르게 후보 문서 집합을 검색하여 다수를 수집한 뒤
 - 후속 단계에서 보다 정교한 알고리즘/모델로 재정렬 및 필터링하는 절차
- 기대 효과
 - 검색 효율성과 정확도라는 Trade-off를 조율
 - 단순 검색 모델로 놓칠 수 있는 문서를 순위 상위로 끌어올려 사용자가 더 정확한 결과를 얻도록



- 대다수 LLM의 처리 가능한 토큰 수 증가
 - LLaMA 3: 128k
 - OpenAI o3-mini: 200k
- 그러나 LLM이 Context 내에서 중요한 정보를 위치에 상관없이 효과적으로 활용할 수 있음을 의미하지는 않음
- 레거시 모델에서 Context 내 중요한 정보의 위치에 따라 답변 성능이 크게 달라지는 현상이 관측됨





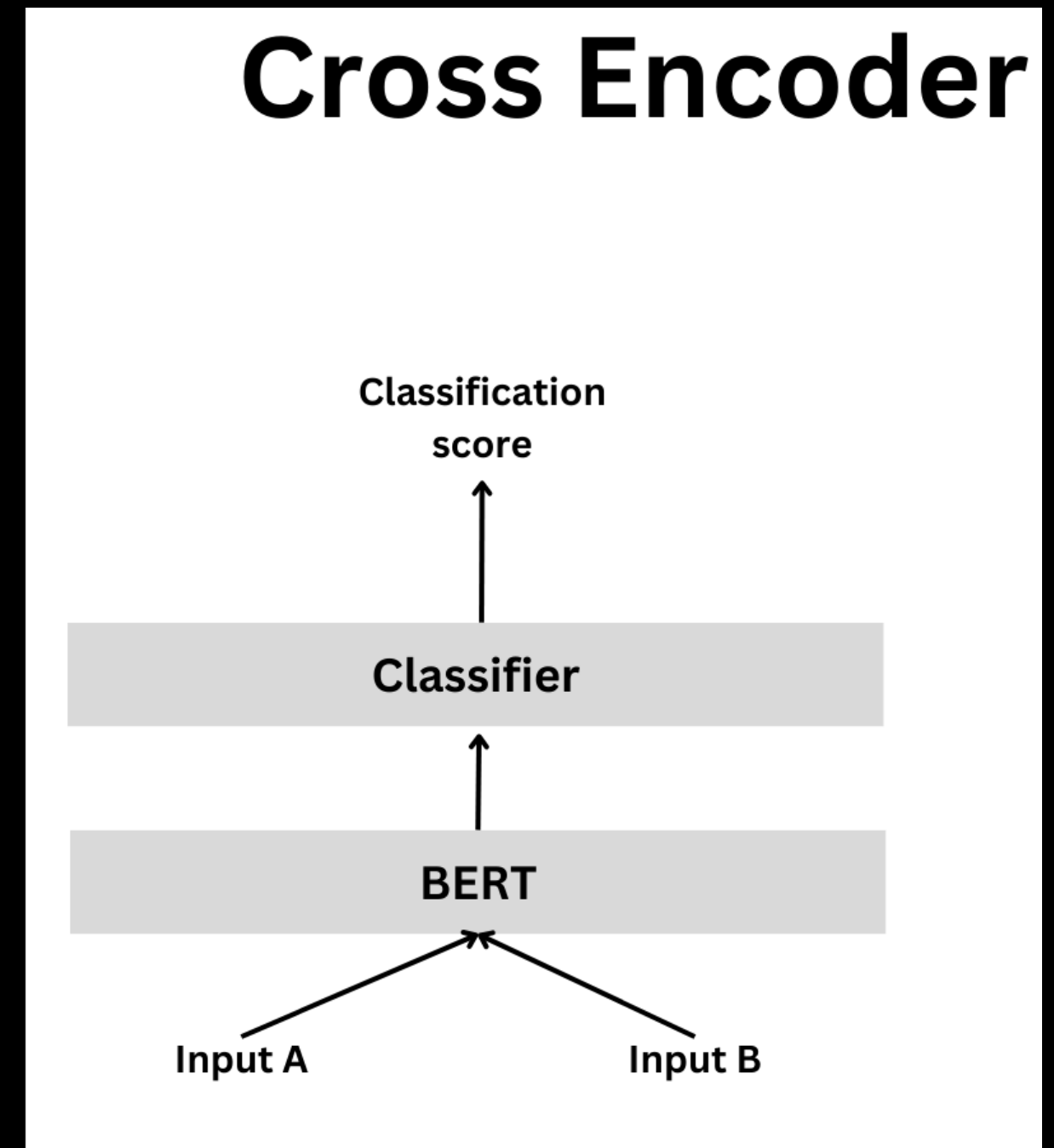
주요 Reranking 알고리즘

- Traditional Learning-to-Rank Reranker
 - RankSVM, LambdaMART
 - 사전 정의된 다양한 Feature을 입력으로 받아 문서의 최종 랭킹 점수를 예측하도록 지도 학습
 - Features: BM25 점수, 문서 길이, 클릭률, 인위 부여된 등급, 신뢰도 등
- 장점
 - ML 기반으로, 매우 가벼워 Real-time inference 속도를 높임
 - 도메인에 특화된 다양한 신호를 통합할 수 있음
- 단점
 - Feature engineering 의존도가 높으며
 - Feature을 Manual로 추출할 수 없어 End-to-end system 구축이 어려움



주요 Reranking 알고리즘

- BERT 기반 Cross-Encoder Reranker
 - 가장 대표적인 방식
 - 쿼리와 문서를 하나의 입력 시퀀스로 함께 Transformer에 투입
 - 관련성 점수를 출력하는 모델
 - 정교한 계산 및 매칭이 가능하나
 - 모든 Chunk에 대하여 Query와 대조해야 하므로 시간과 비용 효율이 낮음

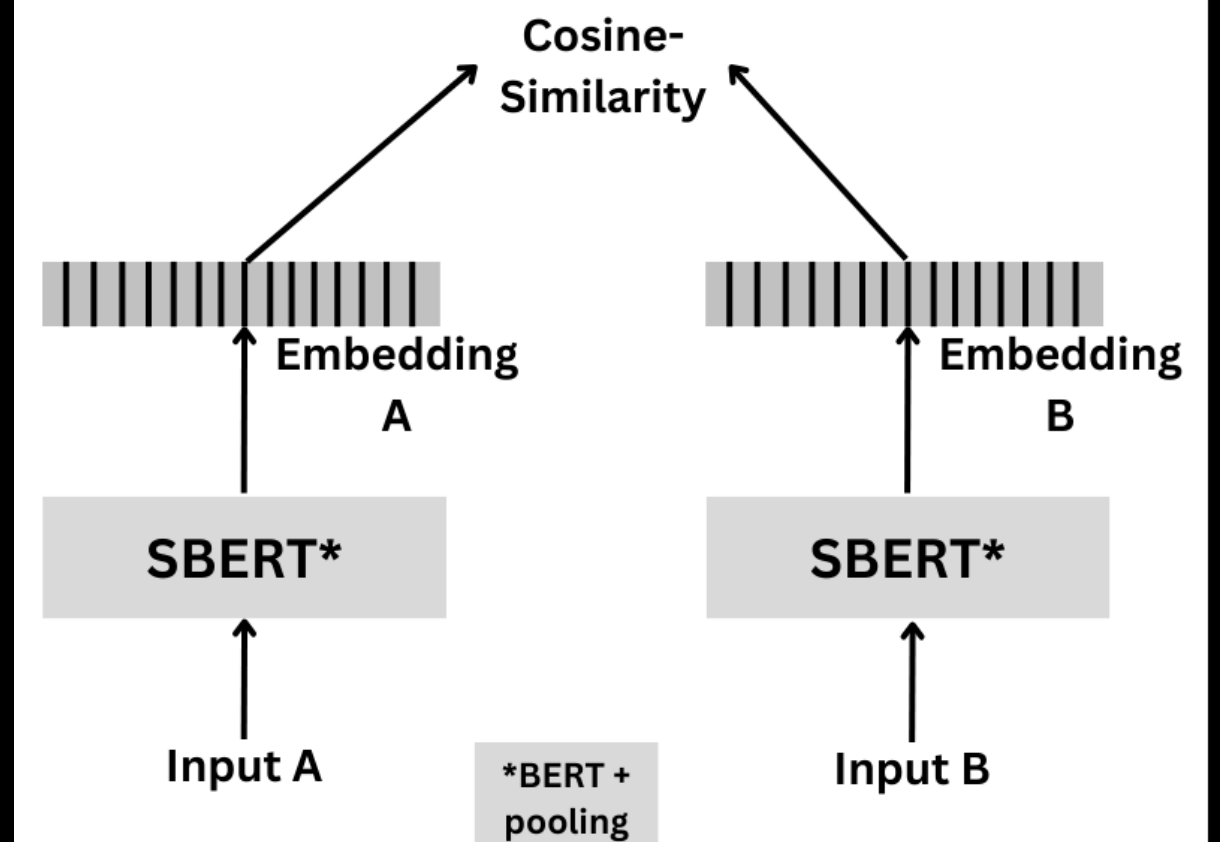




주요 Reranking 알고리즘

- Bi-Encoder 기반 Reranker
 - Query와 Chunk를 각각 별도의 인코더로 임베딩
 - 벡터 공간에서의 유사도로 관련성을 판단
 - 문서 임베딩들을 미리 계산하여 Caching 가능
 - Query 입력 시 임베딩만 계산하여
 - 미리 저장된 Chunk 벡터와 KNN으로 빠르게 검색 가능
 - 다만 Cross-encoder에 비하여 정확도가 떨어짐

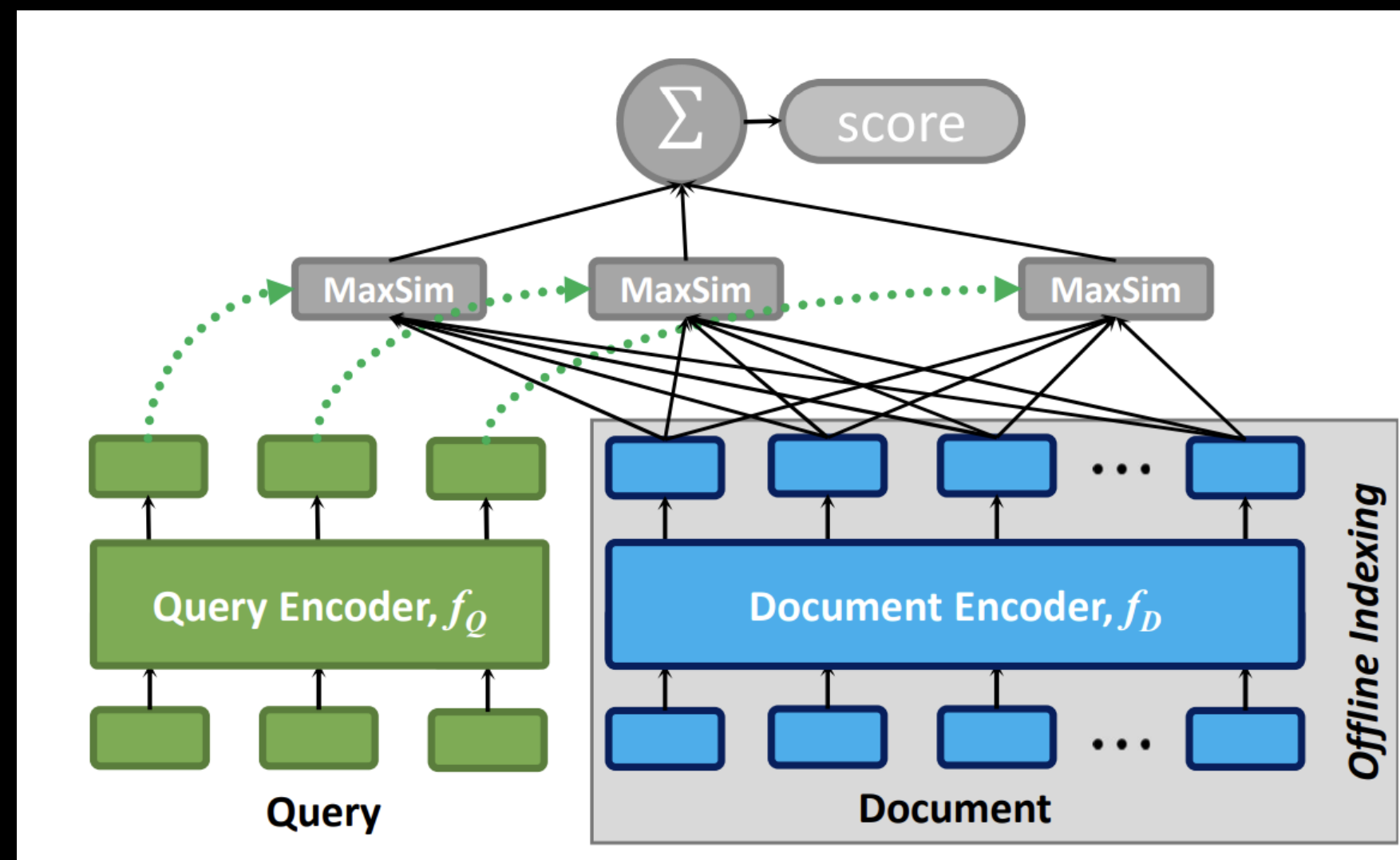
Bi-encoder





주요 Reranking 알고리즘

- Late interaction models
 - Query와 문서를 독립적으로 인코딩하면서도
 - Chunk 하나에 대한 단일 벡터가 아닌, 토큰 마다 임베딩 벡터를 생성하여 다중 벡터 세트 보존
 - Query 또한 토큰 임베딩을 수행 후, Query 토큰과 Chunk 토큰 간 유사도 점수 계산





주요 Reranking 알고리즘

- Late interaction models
 - Query token q_i 에 대한 Chunk token d_j 의 기여도를 점수화하여, 각 Chunk 내 토큰들의 점수 통계가 Query에 대한 Relevance를 의미한다고 간주
 - Cross-encoder에 비해 100배 가량 빠르며, 정확도 차이가 적음
 - 그러나 모든 토큰에 대하여 강도 높은 Embedding 과정이 선행되어야 함
 - 대표적으로 **CoBERT**가 이에 해당하는 모델



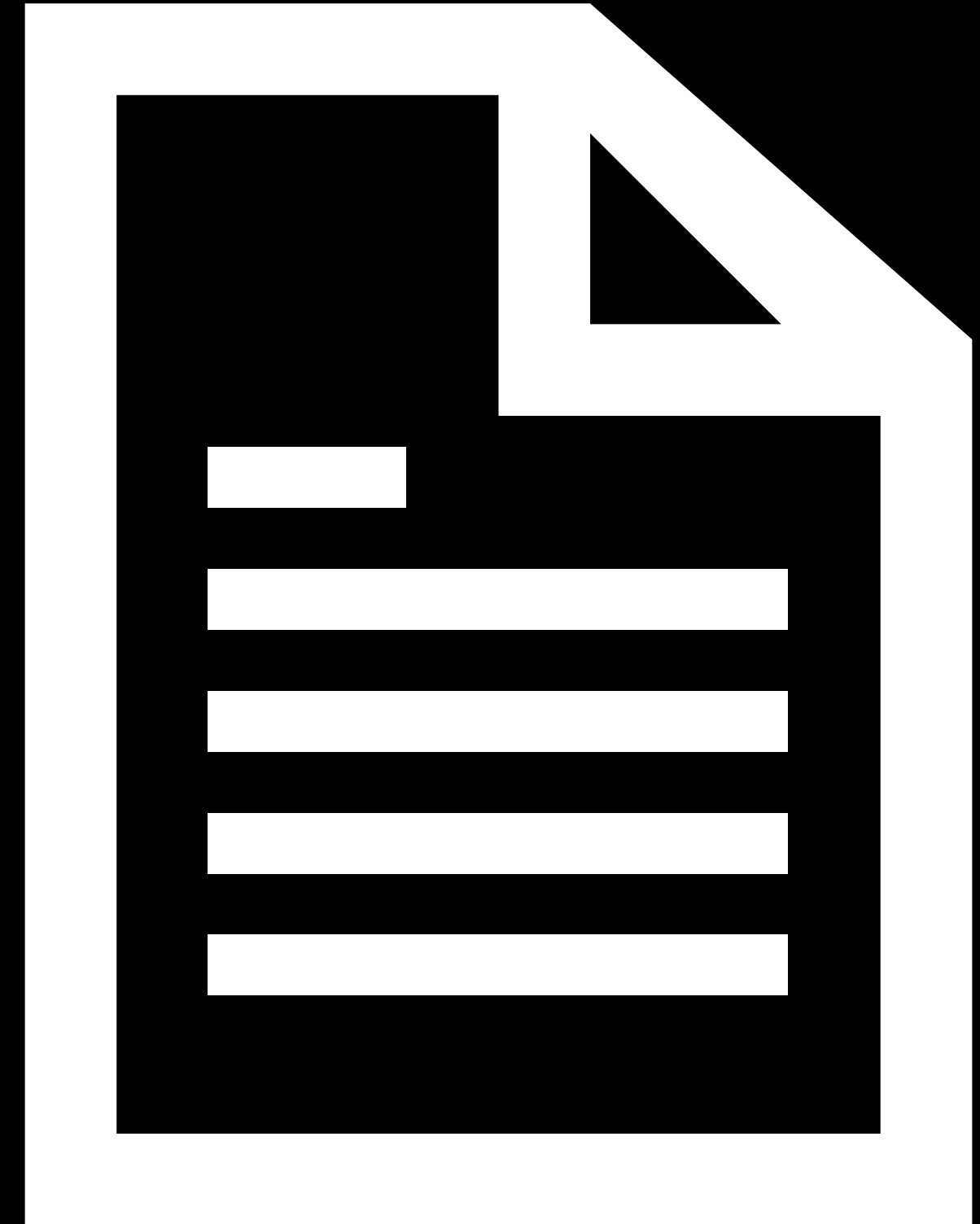
Rerank를 적용한 RAG 챗봇

Rerank와 Modular
RAG

Rerank 기법

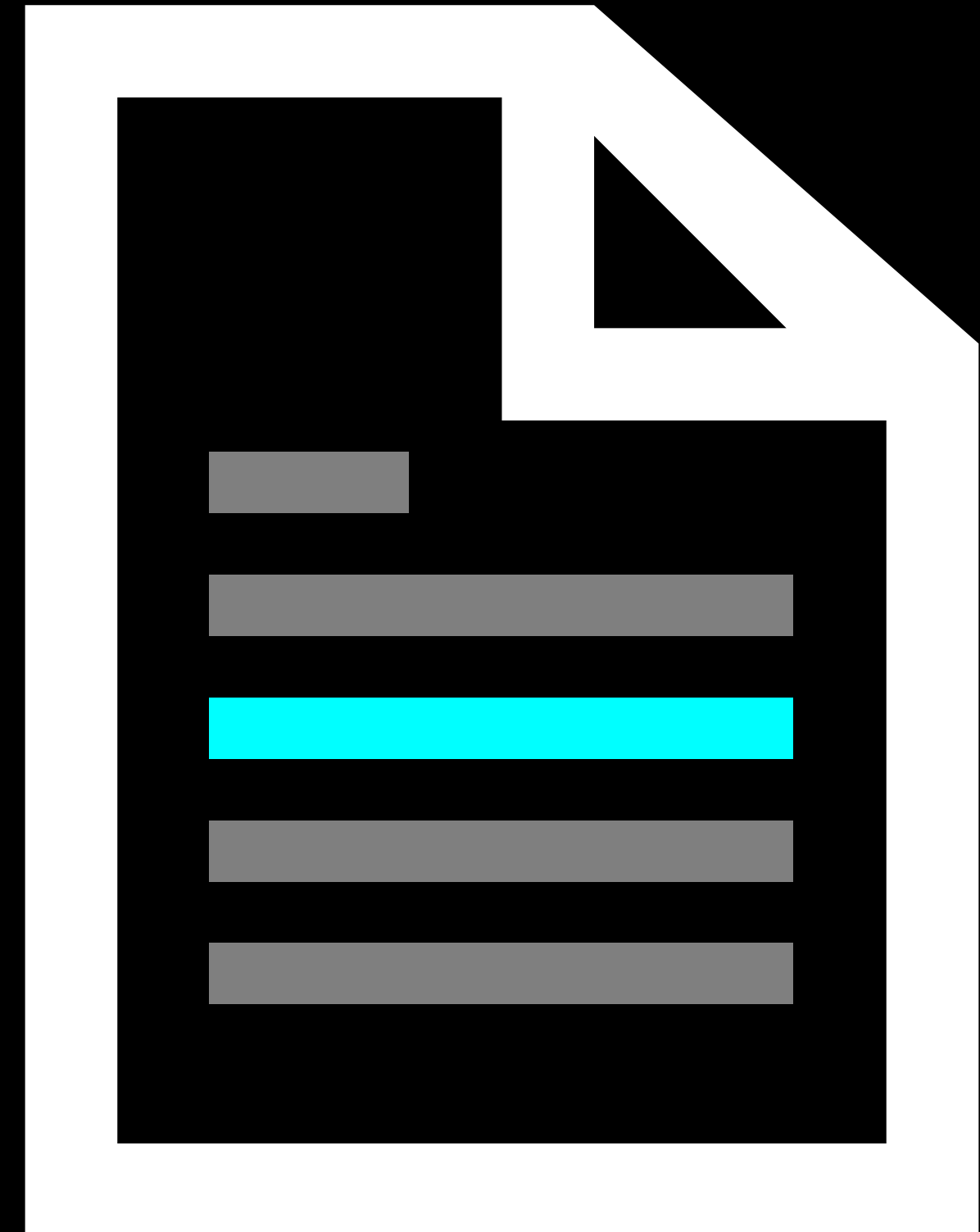
1. Rerank를 적용한
RAG 챗봇

- 이전 실습에서는 문서를 1,000자 단위로 분할하여 사용하고, 가장 **연관성이 높은 문서 1개**를 증강하였다.



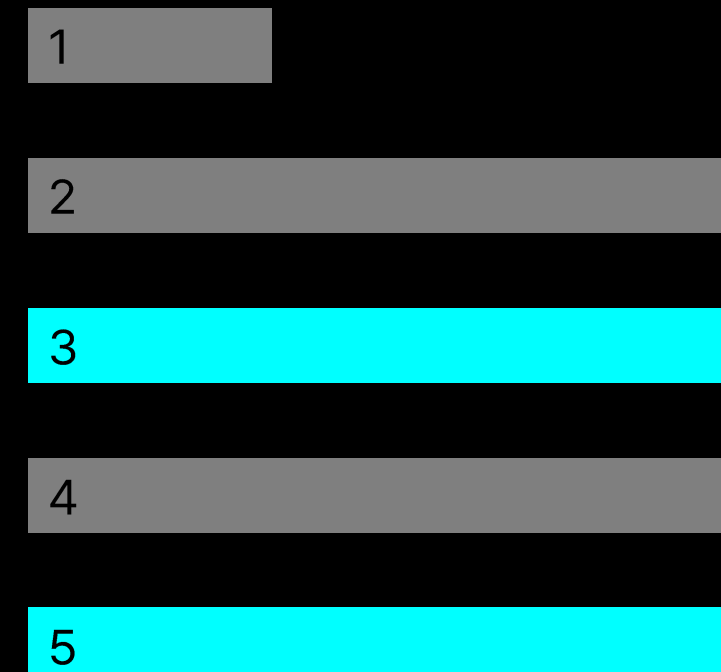


- 이전 실습에서는 문서를 1,000자 단위로 분할하여 사용하고, 가장 **연관성이 높은 문서 1개**를 증강하였다.
- 질문과 관련된 텍스트가 문서의 중간에 위치할 경우, 정보가 컨텍스트의 중심에 놓이게 되어 챗봇의 성능 저하를 초래할 수 있다.





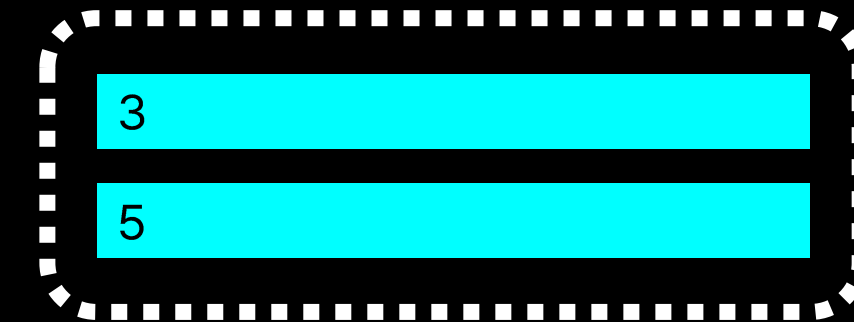
- 이전 실습에서는 문서를 1,000자 단위로 분할하여 사용하고, 가장 **연관성이 높은 문서 1개**를 증강하였다.
- 질문과 관련된 텍스트가 문서의 중간에 위치할 경우, 정보가 컨텍스트의 중심에 놓이게 되어 챗봇의 성능 저하를 초래할 수 있다.
- 이번 실습에서는 문서를 더욱 세분화한 후,





- 이전 실습에서는 문서를 1,000자 단위로 분할하여 사용하고, 가장 **연관성이 높은 문서 1개**를 증강하였다.
- 질문과 관련된 텍스트가 문서의 중간에 위치할 경우, 정보가 컨텍스트의 중심에 놓이게 되어 챗봇의 성능 저하를 초래할 수 있다.
- 이번 실습에서는 문서를 더욱 세분화한 후, Rerank 기법을 활용하여 **연관성이 높은 문서 n개를 연관성 순서대로 결합하여 증강**하는 RAG를 구현한다.

증강할 문서 조각



1

2

4

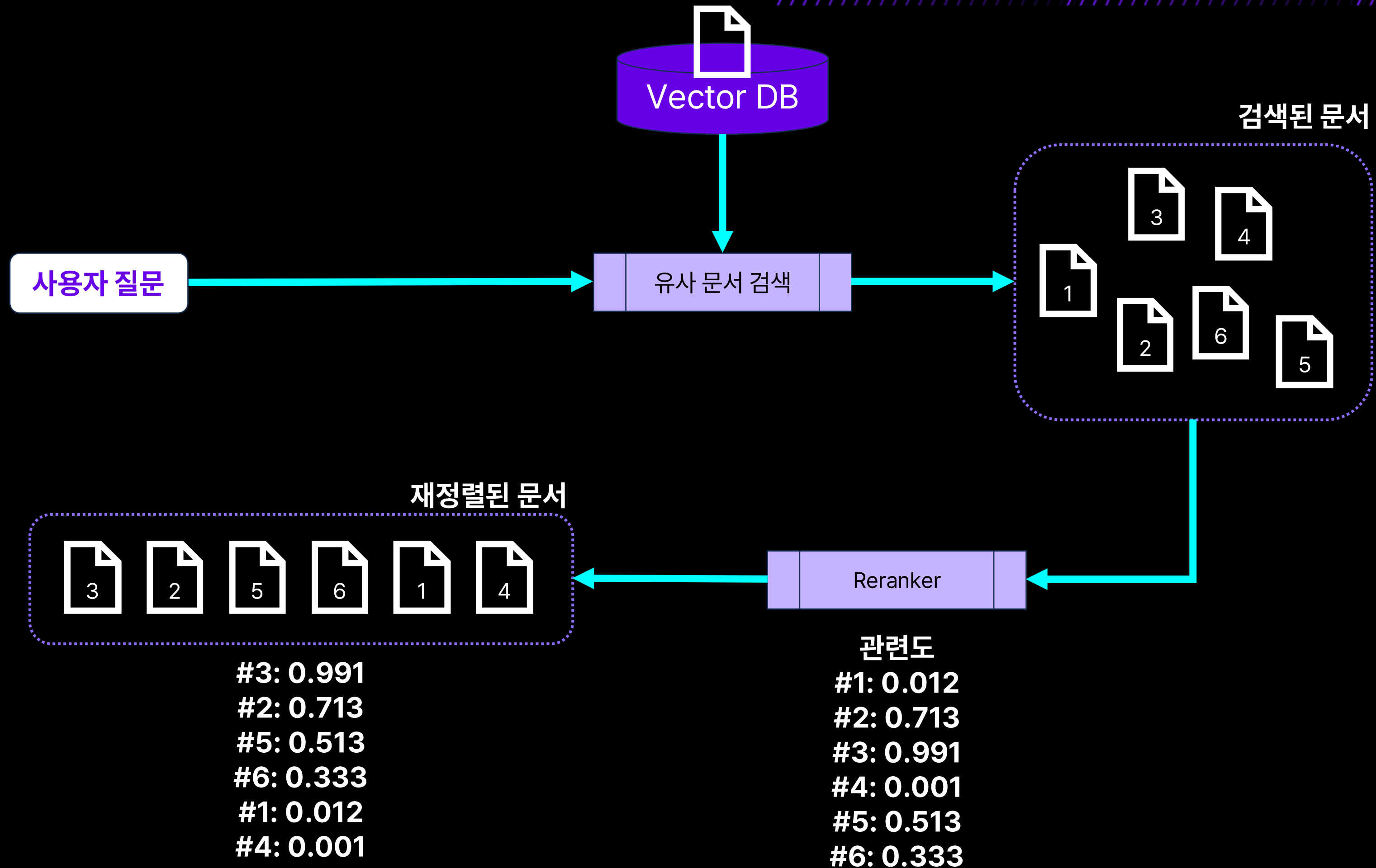


검색 후 재정렬

Rerank와 Modular RAG

Rerank 기법

1. Rerank를 적용한 RAG 챗봇





검색 후 재정렬

Rerank와 Modular RAG

Rerank 기법

1. Rerank를 적용한 RAG 챗봇

